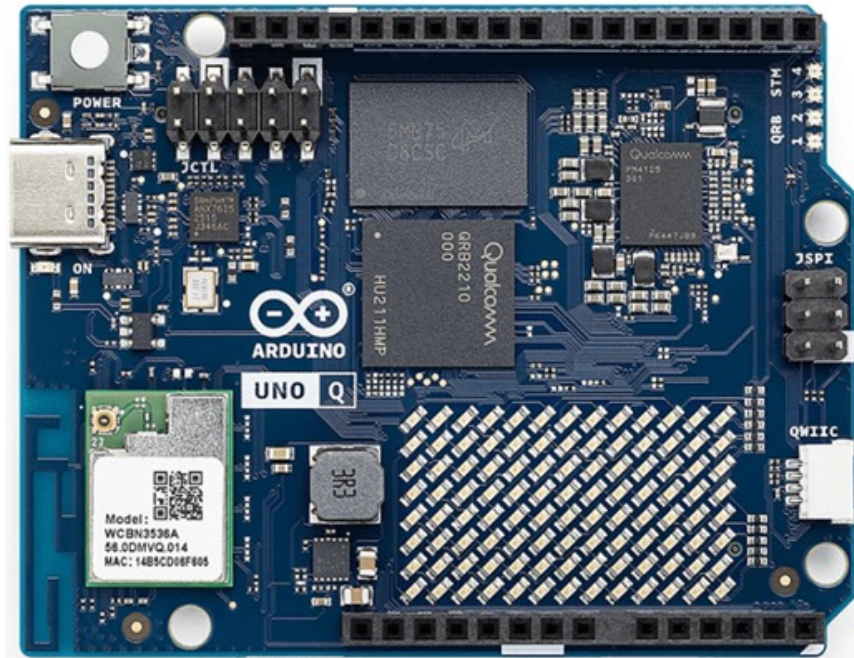
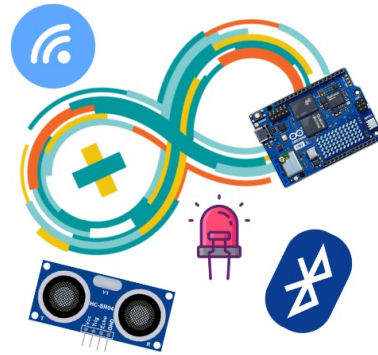
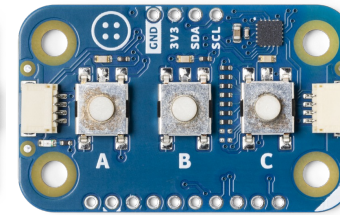
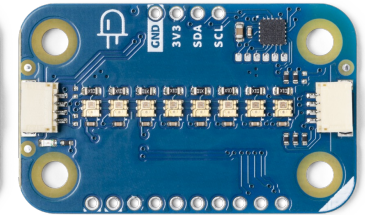


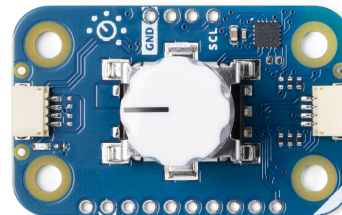
Modulino® **MOVEMENT**



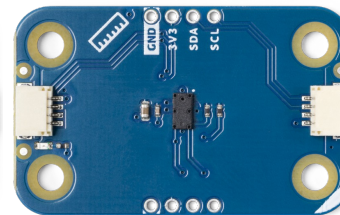
Modulino® **BUTTONS**



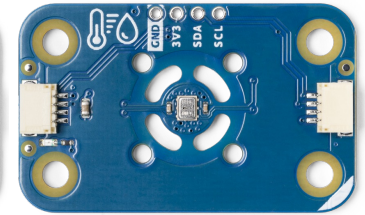
Modulino® **PIXELS**



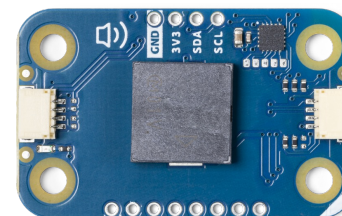
Modulino® **KNOB**



Modulino® **DISTANCE**



Modulino® **THERMO**



Modulino® **BUZZER**

Modulino®

รณ อดมศรโพนุลย
 หองปฏบตการระบบอจจรยและทุนยนตอตโนมต (LISAR)
 คณะศนโนลยสารสนเทศและการสือสาร
 มหาวทยาลยพะเยา

Arduino Modulino®: สร้างสรรค์โปรเจกต์อัจฉริยะได้ง่ายและรวดเร็ว

Arduino Modulino® คือซีรีส์ของบอร์ดโมดูลาร์ขนาดเล็กที่ถอดออกมาเพื่อการสร้างต้นแบบอย่างรวดเร็ว โดยใช้งานร่วมกับคอนเนกเตอร์ Qwiic ซึ่งเป็นมาตรฐานอุตสาหกรรมที่ช่วยให้การเชื่อมต่อเซนเซอร์และอุปกรณ์ต่าง ๆ เป็นเรื่องง่ายโดยไม่ต้องบัดกรี

จุดเด่นของระบบการเชื่อมต่อ Qwiic



เชื่อมต่อแบบ Plug-and-Play

ไม่ต้องใช้เบรด์บอร์ด สายจัมเปอร์ หรือการบัดกรี ช่วยลดความยุ่งยากในการประกอบวงจร



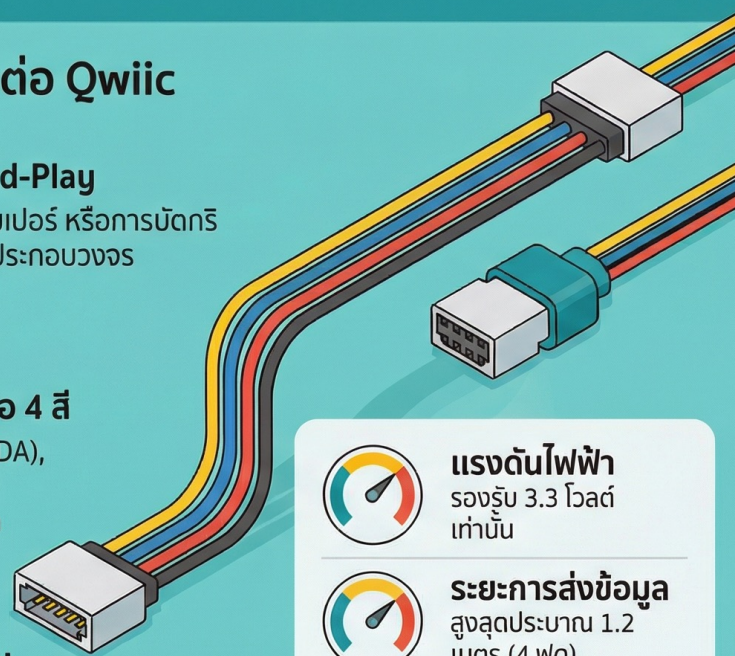
มาตรฐานสายเชื่อมต่อ 4 สี

ใช้สีเหลือง (SCL), น้ำเงิน (SDA), แดง (3.3V) และดำ (GND) เพื่อป้องกันการต่อผิดพลาด



รองรับการต่อพ่วงแบบ Daisy-chain

สามารถเชื่อมต่ออุปกรณ์หลายตัวต่อเรียงกันได้ผ่านบัส I2C เพียงชุดเดียว



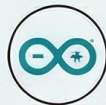
แรงดันไฟฟ้า

รองรับ 3.3 โวลต์ เท่านั้น



ระยะการส่งข้อมูล

สูงสุดประมาณ 1.2 เมตร (4 ฟุต)

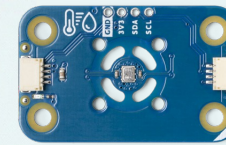


อินเทอร์เฟซ

I2C (Bus 2 / Wire1 บน Arduino UNO Q)

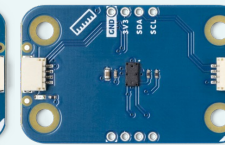
ทำความรู้จักโมดูล Modulino®

โมดูลตรวจสอบสภาพแวดล้อมและการเคลื่อนไหว



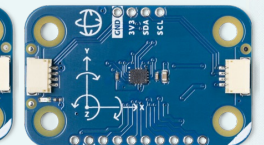
Thermo

ไม่ต้องใช้เบรด์บอร์ด (อุณหภูมิ/ความชื้น)



Distance

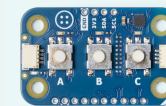
(ระยะทาง ToF) (wav →)



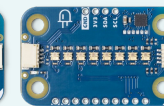
Movement

acceleration/rotation (ความเร่ง/การหมุน)

โมดูลควบคุมและแสดงผล



Buttons

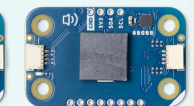


Pixels

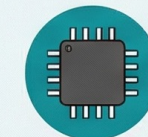
RGB LEDs



Knob



Buzzer



ควบคุมด้วย MCU ประสิทธิภาพสูง

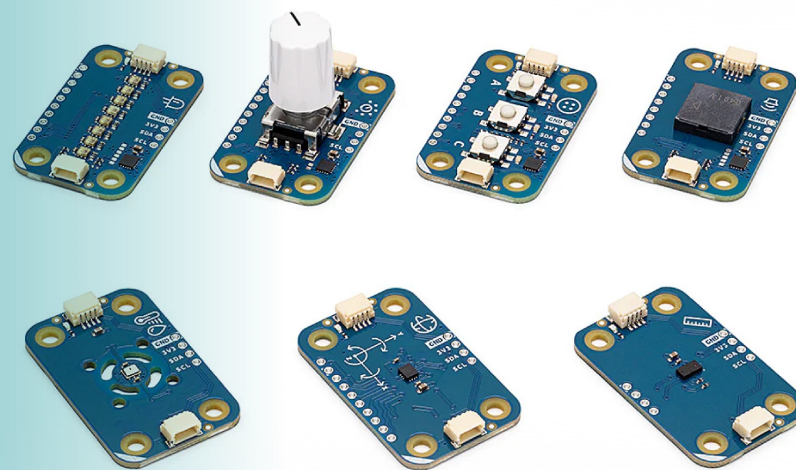
โมดูลส่วนใหญ่ใช้ไมโครคอนโทรลเลอร์ STM32C011F4 ในการประมวลผลภายในบอร์ด

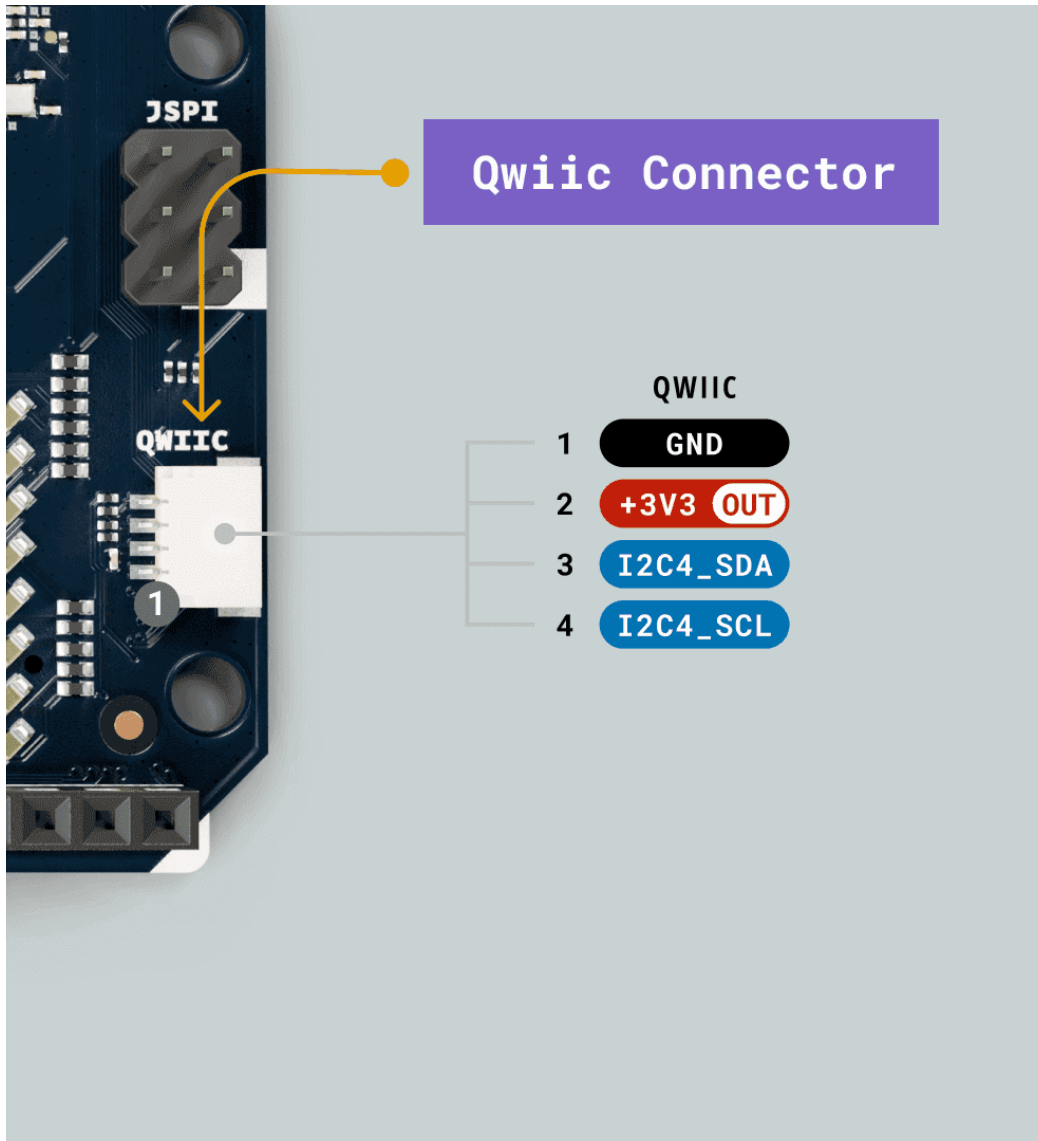
UNO

Q

ARDUINO Modulino®

ปลดล็อกความเป็นไปได้ใหม่ ๆ ทางความคิด
สร้างสรรค์ด้วย Arduino Modulinos
ชุดบอร์ดขนาดเล็กจัดแบบโมดูลาร์ที่ยืดหยุ่น
ออกแบบมาสำหรับการสร้างต้นแบบอย่างรวดเร็ว
และการผสานรวมเข้ากับโปรเจกต์ได้อย่างไร
ร่อยต่อ สร้างสรรค์งานได้ฉลาดขึ้น เร็วขึ้น และ
สร้างสรรค์มากขึ้น ด้วยโซลูชันพร้อมใช้งานที่ตอบ
โจทย์ทุกไอเดีย ไม่ว่าจะเล็กหรือใหญ่





บอร์ด **Arduino UNO Q** มาพร้อมกับ
คอนเนกเตอร์ **Qwiic** บนบอร์ด ซึ่งเป็น
โซลูชันที่เรียบง่ายและไม่ต้องใช้เครื่องมือ
สำหรับเชื่อมต่ออุปกรณ์แบบ I²C
ระบบนิเวศ **Qwiic** ซึ่งพัฒนาโดย
SparkFun Electronics ได้กลายเป็น
มาตรฐานในอุตสาหกรรมสำหรับการสร้าง
ต้นแบบอย่างรวดเร็วด้วยอุปกรณ์ I²C โดย
ช่วยให้คุณสามารถเชื่อมต่อเซนเซอร์
จอแสดงผล และอุปกรณ์ต่อพ่วงอื่น ๆ ได้
โดยไม่ต้องบัดกรีหรือเดินสายที่ซับซ้อน

คอนเนกเตอร์ Qwiic บนบอร์ด Arduino UNO Q เชื่อมต่อ
อยู่กับบัส I²C ตัวที่สอง (I²C4) ซึ่งใช้งานผ่านอ็อบเจกต์
Wire1 แทนที่จะเป็น Wire
และ คอนเนกเตอร์ Qwiic รองรับแรงดันไฟ 3.3 โวลต์
เท่านั้น

Qwiic – โครงสร้างสาย (Anatomy of a Cable)

สาย Qwiic ถูกออกแบบให้ใช้งานง่ายและเป็นมาตรฐานเดียวกันทุกเส้น เพื่อให้การเชื่อมต่ออุปกรณ์ I2C ทำได้รวดเร็วและลดความผิดพลาด

ระยะการส่งข้อมูล

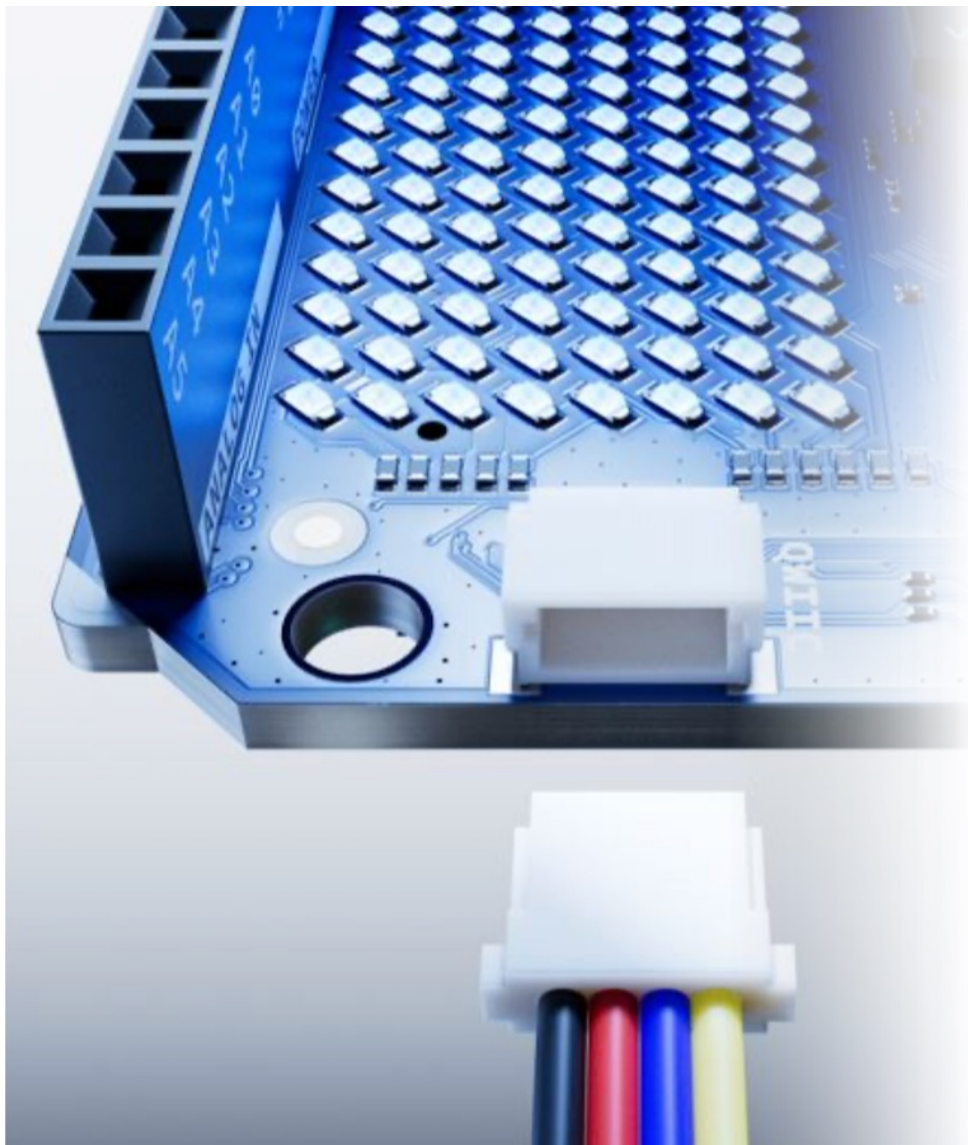
รองรับการส่งข้อมูลได้ไกลสูงสุดประมาณ 4 ฟุต (ประมาณ 1.2 เมตร) หากต้องการขยายบัส I2C ให้ไกลกว่านี้ สามารถใช้โมดูล Multiplexer เพื่อช่วยขยายระบบได้



มาตรฐานสีของสาย Qwiic

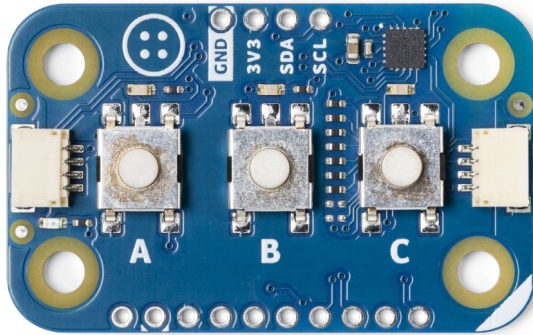
- สีเหลือง (Yellow) – SCL
- สีน้ำเงิน (Blue) – SDA
- สีแดง (Red) – 3.3V
- สีดำ (Black) – GND

การกำหนดสีแบบตายตัวนี้ช่วยลดความสับสนและป้องกันการต่อสายผิดพลาด



ข้อดีของระบบ Qwiic

- 🔌 **การเชื่อมต่อแบบ Plug-and-Play:** ไม่ต้องใช้เบรด์บอร์ด สายจัมเปอร์ หรือการบัดกรี
- ➔ **หัวต่อแบบกำหนดทิศทาง (Polarized connectors):** ป้องกันการเสียบกลับด้านโดยไม่ตั้งใจ
- 🔗 **รองรับการต่อพ่วงแบบ Daisy-chain:** สามารถเชื่อมต่ออุปกรณ์หลายตัวต่อเรียงกันได้
- ⚡ **มีตัวต้านทาน Pull-up ในตัว:** ไม่จำเป็นต้องใช้ตัวต้านทานภายนอกเพิ่มเติม
- 📏 **มาตรฐานขาเชื่อมต่อเดียวกัน (Standard pinout):** ใช้งานร่วมกันได้กับอุปกรณ์ทั้งหมดในระบบนิเวศ Qwiic



Modulino® **BUTTONS**

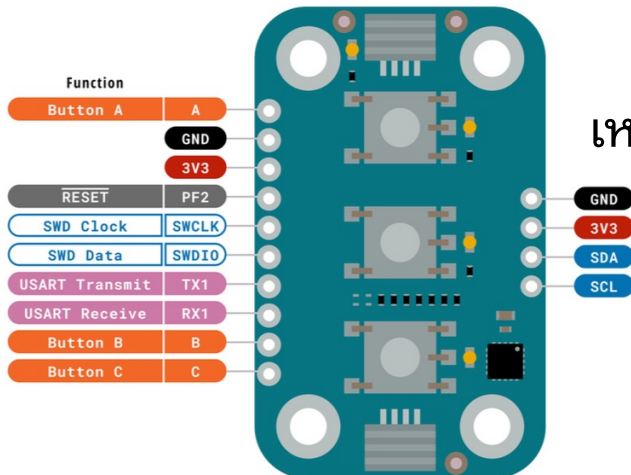
Modulino® Buttons เป็นโมดูลปุ่มกดอัจฉริยะในซีรีส์ Modulino® ที่ถูกออกแบบมาเพื่อเพิ่มประสิทธิภาพในการโต้ตอบกับผู้ใช้ (User Interaction) ในระดับที่ซับซ้อนกว่าปุ่มกดทั่วไป

Modulino® Buttons ใช้ไมโครคอนโทรลเลอร์ **STM32C011F4** บนบอร์ด และมาพร้อมปุ่มกดแบบ **Single Pole Single Throw (SPST)** จำนวน 3 ปุ่ม พร้อมไฟแสดงสถานะ (Indicator LEDs) จำนวน 3 ดวง ชุดอุปกรณ์นี้รองรับทั้งการอ่านสัญญาณดิจิทัลแบบง่ายผ่าน **I²C** และการเชื่อมต่อหรือโปรแกรมใหม่ในระดับที่ซับซ้อนมากขึ้น เหมาะสำหรับโปรเจกต์ที่ต้องการการโต้ตอบกับผู้ใช้ การนำทางเมนู หรือการสั่งงานควบคุมอย่างรวดเร็ว

ค่า Default I2C Address คือ **0x7C**

เหมาะอย่างยิ่งสำหรับ:

- การสร้างระบบนำทางเมนู (Menu navigation)
- โปรเจกต์ที่ต้องการการโต้ตอบกับผู้ใช้อย่างรวดเร็ว
- การใช้เป็นชุดสั่งงานควบคุม (Command control) ในโปรเจกต์ IoT หรือหุ่นยนต์



ฟังก์ชันหลัก (Key Functions)

- `update()` : ขอข้อมูลล่าสุดจากโมดูลปุ่มกด
- `isPressed(index)` : ตรวจสอบว่าปุ่มที่ระบุถูกกดอยู่หรือไม่ (0 = ปุ่ม A, 1 = ปุ่ม B, 2 = ปุ่ม C)
- `setLeds(A, B, C)` : กำหนดสถานะของไฟ LED แต่ละดวง (true = เปิด, false = ปิด)

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

📖 MsgPack 0.4.2

```
#include "Arduino_Modulino.h" // เรียกใช้ไลบรารีสำหรับควบคุมอุปกรณ์ตระกูล Modulino
#include <Arduino_RouterBridge.h> // เรียกใช้ไลบรารีสำหรับการใช้คำสั่ง Monitor

ModulinoButtons buttons; // สร้างอ็อบเจกต์ชื่อ buttons สำหรับควบคุม Modulino Buttons

void setup() {
    Monitor.begin(); // เริ่มต้น Monitor สำหรับแสดงข้อความบนคอนโซลฝั่ง Arduino C++
    Modulino.begin(); // เริ่มต้นระบบ Modulino (เตรียมการสื่อสาร I2C/Qwiic)
    buttons.begin(); // เริ่มต้นโมดูลปุ่มกด
    buttons.setLeds(true, true, true); // สั่งเปิดไฟ LED ทั้ง 3 ดวงบนโมดูลปุ่ม (A, B, C) โดย true = เปิด, false = ปิด
}

void loop() {
    if (buttons.update()) { // ขอข้อมูลสถานะล่าสุดจากโมดูลปุ่ม update() จะคืนค่า true เมื่ออ่านข้อมูลสำเร็จ
        if (buttons.isPressed(0)) { // ตรวจสอบว่าปุ่ม A (index 0) ถูกกดหรือไม่
            Monitor.println("ปุ่ม A ถูกกด!");
        }

        else if (buttons.isPressed(1)) { // ตรวจสอบว่าปุ่ม B (index 1) ถูกกดหรือไม่
            Monitor.println("ปุ่ม B ถูกกด!");
        }

        else if (buttons.isPressed(2)) { // ตรวจสอบว่าปุ่ม C (index 2) ถูกกดหรือไม่
            Monitor.println("ปุ่ม C ถูกกด!");
        }
    }
}
```


ดัดแปลงให้ LED ติดเฉพาะปุ่มที่กด

▼ Sketch Libraries

Arduino_Modulino 0.7.0

MsgPack 0.4.2

🔍 ความหมาย

- ใช้ทุกปุ่มแยกกัน
- ถ้ากดหลายปุ่มพร้อมกัน → จะแสดงหลายข้อความ

📌 ตัวอย่าง

ถ้ากด A + B พร้อมกัน

👉 จะแสดง

ปุ่ม A ถูกกด!

ปุ่ม B ถูกกด!

```
#include "Arduino_Modulino.h"
#include <Arduino_RouterBridge.h>

ModulinoButtons buttons;

void setup() {
  Monitor.begin();
  Modulino.begin();
  buttons.begin();

  // เริ่มต้นให้ไฟทั้งหมดดับก่อน
  buttons.setLeds(false, false, false);
}

void loop() {

  // ขอข้อมูลล่าสุดจากโมดูล
  if (buttons.update()) {

    // อ่านสถานะปุ่มแต่ละตัว
    bool a = buttons.isPressed(0);
    bool b = buttons.isPressed(1);
    bool c = buttons.isPressed(2);

    // สั่งให้ LED ติดเฉพาะปุ่มที่ถูกกด
    buttons.setLeds(a, b, c);

    // แสดงข้อความเมื่อมีการกด
    if (a) {
      Monitor.println("ปุ่ม A ถูกกด!");
    }
    if (b) {
      Monitor.println("ปุ่ม B ถูกกด!");
    }
    if (c) {
      Monitor.println("ปุ่ม C ถูกกด!");
    }
  }
}
```

```
#include "Arduino_Modulino.h"
#include <Arduino_RouterBridge.h>

ModulinoButtons buttons;

void setup() {
  Monitor.begin();
  Modulino.begin();
  buttons.begin();

  // เริ่มต้นให้ไฟทั้งหมดดับก่อน
  buttons.setLeds(false, false, false);
}

void loop() {

  // ขอข้อมูลล่าสุดจากโมดูล
  if (buttons.update()) {

    // อ่านสถานะปุ่มแต่ละตัว
    bool a = buttons.isPressed(0);
    bool b = buttons.isPressed(1);
    bool c = buttons.isPressed(2);

    // สั่งให้ LED ติดเฉพาะปุ่มที่ถูกกด
    buttons.setLeds(a, b, c);

    // แสดงข้อความเมื่อมีการกด
    if (a) {
      Monitor.println("ปุ่ม A ถูกกด!");
    }
    else if (b) {
      Monitor.println("ปุ่ม B ถูกกด!");
    }
    else if (c) {
      Monitor.println("ปุ่ม C ถูกกด!");
    }
  }
}
```

🔍 ความหมาย

- ถ้า A ถูกกด → แสดงข้อความ A
- ถ้า A ไม่ถูกกด แต่ B ถูกกด → แสดงข้อความ B
- ถ้า A และ B ไม่ถูกกด แต่ C ถูกกด → แสดงข้อความ C
- ถ้ากดหลายปุ่มพร้อมกัน → จะแสดงแค่ปุ่มแรกๆ ที่เข้าเงื่อนไขเท่านั้น

📌 ตัวอย่าง

ถ้ากด A + B พร้อมกัน

👉 จะแสดงแค่ "ปุ่ม A ถูกกด!"

▼ Sketch Libraries

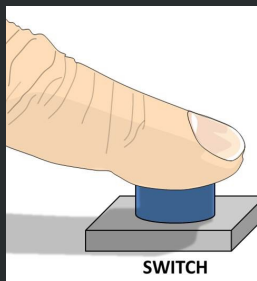
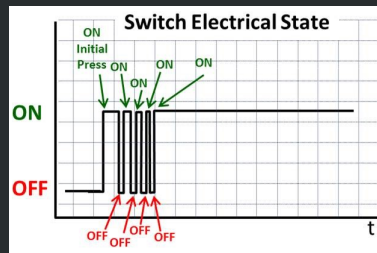
- 📖 Arduino_Modulino 0.7.0
- 📖 MsgPack 0.4.2
- 📖 Button2 2.5.0

```
#include <Arduino_RouterBridge.h> // เรียกใช้ไลบรารีสำหรับระบบ RouterBridge เพื่อใช้กับคำสั่ง Monitor
#include "Arduino_Modulino.h" // เรียกใช้ไลบรารีสำหรับควบคุมอุปกรณ์ตระกูล Modulino
#include "Button2.h" // เรียกใช้ไลบรารี Button2 สำหรับจัดการ event การกดปุ่มหลายรูปแบบ
```

```
Button2 button; // สร้างอ็อบเจกต์ button สำหรับจัดการ event ปุ่ม
ModulinoButtons modulino_buttons; // สร้างอ็อบเจกต์สำหรับควบคุม Modulino Buttons
String msg; // ตัวแปรเก็บข้อความสำหรับแสดงผล
```

```
void setup() {
  Monitor.begin(); // เริ่มต้นคำสั่ง Monitor
  Modulino.begin(); // เริ่มต้นโมดูล Modulino
  modulino_buttons.begin(); // เริ่มต้นโมดูลปุ่ม
  // ตั้งค่า debounce time = 35 ms (กันสัญญาณสั่นจากปุ่ม)
  button.setDebounceTime(35);
  // กำหนดฟังก์ชันสำหรับอ่านสถานะปุ่ม (ใช้แทน digitalRead)
  button.setButtonStateFunction(button0StateHandler);
  // กำหนด handler สำหรับคลิกครั้งเดียว
  button.setClickHandler(handler);
  // กำหนด handler สำหรับดับเบิลคลิก
  button.setDoubleClickHandler(handler);
  // กำหนด handler สำหรับทริปเปิ้ลคลิก
  button.setTripleClickHandler(handler);
  // กำหนด handler สำหรับกดค้าง (long click)
  button.setLongClickHandler(handler);
  // เริ่มต้นใช้งานปุ่มแบบ virtual pin
  button.begin(BTN_VIRTUAL_PIN);
}
```

```
void loop() {
  // ต้องเรียก loop() ของ Button2 ตลอดเวลา
  // เพื่อให้ระบบตรวจจับ event ได้
  button.loop();
}
```



```
// ฟังก์ชันนี้ใช้ให้ Button2 อ่านสถานะปุ่มจาก Modulino แทนการอ่าน GPIO ปกติ
uint8_t button0StateHandler() {
  modulino_buttons.update(); // อัปเดตข้อมูลล่าสุดจากโมดูลปุ่ม
  // ถ้าปุ่มหมายเลข 0 ถูกกด → ส่งค่า LOW
  // ถ้าไม่ถูกกด → ส่งค่า HIGH
  // (Button2 ใช้ logic แบบ active LOW)
  return modulino_buttons.isPressed(0) ? LOW : HIGH;
}

// ฟังก์ชัน handler จะถูกเรียกเมื่อเกิด event การกดปุ่ม
void handler(Button2& btn) {
  String msg_0 = " "; // ตัวแปรเก็บประเภทของการกด
  // ตรวจสอบชนิดของ event (คลิก, ดับเบิลคลิก ฯลฯ)
  switch (btn.getType()) {
    case single_click:
      msg_0 = "กดคลิก ";
      break;
    case double_click:
      msg_0 = "ดับเบิลคลิก ";
      break;
    case triple_click:
      msg_0 = "ทริปเปิ้ลคลิก ";
      break;
    case long_click:
      msg_0 = "กดแช่ ";
      break;
  }

  String msg_1 = " ( "; // สร้างข้อความเพิ่มเติมแสดงจำนวนครั้งที่คลิก
  // แปลงจำนวนครั้งที่คลิกเป็น String
  String msg_2 = String(btn.getNumberOfClicks());
  // ปิดวงเล็บ
  String msg_3 = ") "; // เก็บจำนวนการคลิกปุ่มเช่น 1, 2, หรือ 3 เป็น int
  // รวมข้อความทั้งหมดเข้าด้วยกัน
  msg = msg_0 + msg_1 + msg_2 + msg_3;
  // แสดงผลข้อความผ่าน Monitor
  Monitor.println(msg);
}
```

ถ้าต้องการเพิ่มการใช้งาน ปุ่มที่ 1 (isPressed(1)) และ ปุ่มที่ 2 (isPressed(2)) **มี 2 แนวทางหลัก** ขึ้นอยู่กับว่าอยากให้ปุ่มทำงานแยกกัน หรือให้ปุ่มไหนกดก็ถือว่ากดเหมือนกัน

✅ กรณีที่ 1: ใช้ Button2 แยก 3 ตัว

ถ้าต้องการให้ปุ่ม 0, 1, 2 แยก event กันชัดเจน ควรสร้าง Button2 3 ตัว

🔧 1 ประกาศตัวแปรเพิ่ม
จากเดิม:

```
Button2 button;
```

แก้เป็น:

```
Button2 button0;  
Button2 button1;  
Button2 button2;
```

หมายเหตุ จากโปรแกรมบรรทัดนี้

```
return modulino_buttons.isPressed(0) ? LOW : HIGH;
```

สามารถเขียนให้อยู่ในรูปแบบ if else ได้

```
if (modulino_buttons.isPressed(0)) {  
    return LOW;  
} else {  
    return HIGH;  
}
```


🔧 2 แก้ setup()

จากเดิม:

```
button.setButtonStateFunction(buttonStateHandler);
button.setClickHandler(handler);
button.setDoubleClickHandler(handler);
button.setTripleClickHandler(handler);
button.setLongClickHandler(handler);
button.begin(BTN_VIRTUAL_PIN);
```

แก้เป็น:

```
// --- BUTTON 0 ---
button0.setButtonStateFunction(button0StateHandler);
button0.setClickHandler(handler);
button0.setDoubleClickHandler(handler);
button0.setTripleClickHandler(handler);
button0.setLongClickHandler(handler);
button0.begin(BTN_VIRTUAL_PIN);

// --- BUTTON 1 ---
button1.setButtonStateFunction(button1StateHandler);
button1.setClickHandler(handler);
button1.setDoubleClickHandler(handler);
button1.setTripleClickHandler(handler);
button1.setLongClickHandler(handler);
button1.begin(BTN_VIRTUAL_PIN);

// --- BUTTON 2 ---
button2.setButtonStateFunction(button2StateHandler);
button2.setClickHandler(handler);
button2.setDoubleClickHandler(handler);
button2.setTripleClickHandler(handler);
button2.setLongClickHandler(handler);
button2.begin(BTN_VIRTUAL_PIN);
```

🔧 3 เพิ่ม state handler ของแต่ละปุ่ม

จากเดิม:

```
uint8_t button0StateHandler() {
    modulino_buttons.update();
    return modulino_buttons.isPressed(0) ? LOW : HIGH;
}
```

แก้เป็น:

```
uint8_t button0StateHandler() {
    modulino_buttons.update();
    return modulino_buttons.isPressed(0) ? LOW : HIGH;
}

uint8_t button1StateHandler() {
    modulino_buttons.update();
    return modulino_buttons.isPressed(1) ? LOW : HIGH;
}

uint8_t button2StateHandler() {
    modulino_buttons.update();
    return modulino_buttons.isPressed(2) ? LOW : HIGH;
}
```

4 แก้ loop()

จากเดิม:

```
button.loop();
```

แก้เป็น:

```
void loop() {  
  button0.loop();  
  button1.loop();  
  button2.loop();  
}
```

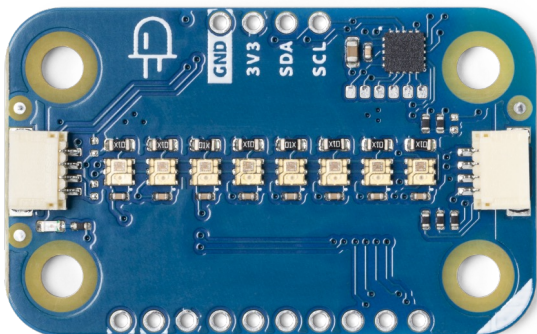
✓ กรณีที่ 2: ให้ปุ่มไหนก็ได้กดแล้วถือว่าเป็นปุ่มเดียวกัน

ถ้าอยากให้ปุ่ม 0,1,2 ทำงานเหมือนกันหมดแค่ buttonStateHandler() แบบนี้:
จากเดิม:

```
uint8_t button0StateHandler() {  
    modulino_buttons.update();  
    return modulino_buttons.isPressed(0) ? LOW : HIGH;  
}
```

แก้เป็น:

```
uint8_t buttonStateHandler() {  
    modulino_buttons.update();  
  
    if (modulino_buttons.isPressed(0) ||  
        modulino_buttons.isPressed(1) ||  
        modulino_buttons.isPressed(2)) {  
        return LOW;  
    }  
  
    return HIGH;  
}
```

Modulino® Pixels เป็นโมดูลไฟแสดงผลอัจฉริยะในซีรีส์ Modulino® ที่ออกแบบมาเพื่อการสร้างเอฟเฟกต์แสงสีและแอนิเมชันในโปรเจกต์ต่าง ๆ

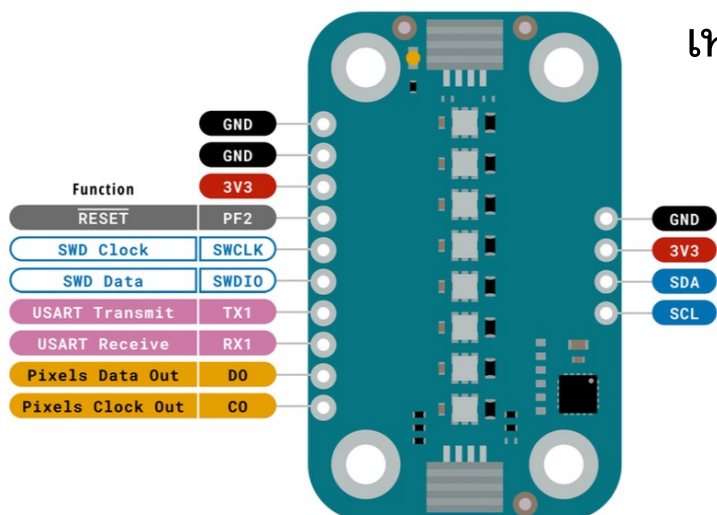
Modulino® Pixels มาพร้อมไฟ **RGB LED** รุ่น LC8822-2020 จำนวน 8 ดวง ควบคุมด้วยไมโครคอนโทรลเลอร์ **STM32C011F4** บนบอร์ดชุดอุปกรณ์นี้รองรับการเชื่อมต่อแบบ I²C ที่ใช้งานง่าย ช่วยให้สามารถควบคุมเอฟเฟกต์แสงสี แอนิเมชัน หรือไฟแสดงสถานะต่าง ๆ ได้อย่างสะดวก เหมาะสำหรับโปรเจกต์หลากหลายประเภท

Modulino® **PIXELS**

ค่าเริ่มต้น (Default) คือ **0x6C**

เหมาะอย่างยิ่งสำหรับ:

- การสร้างเอฟเฟกต์แสงสี (Light effects) และแอนิเมชันรูปแบบต่าง ๆ
- ใช้เป็นไฟแสดงสถานะ (Status indicators) ของระบบหรือเซนเซอร์ตัวอื่น ๆ
- การนำไปผสานรวมกับโปรเจกต์ IoT หรือเครื่องใช้ไฟฟ้าอัจฉริยะที่ต้องการการตอบสนองทางสายตา



ฟังก์ชันหลัก (Key Functions)

- **set(index, color, brightness):**
กำหนดค่าให้ LED ดวงที่ระบุ (หมายเลข 0–7) ให้แสดงสีที่ต้องการ พร้อมกำหนดความสว่าง
- **show():**
สั่งให้แสดงผลการเปลี่ยนแปลงทั้งหมดไปยัง LED (ทำให้การตั้งค่าที่กำหนดไว้แสดงผลจริง)

สีที่สามารถใช้งานได้ (Available colours):

- **RED** = สีแดง
- **BLUE** = สีน้ำเงิน
- **GREEN** = สีเขียว
- **VIOLET** = สีม่วง
- **WHITE** = สีขาว

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

```
// เรียกใช้ไลบรารีสำหรับควบคุมอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// สร้างอ็อบเจกต์สำหรับควบคุม Modulino Pixels (LED 8 ดวง)
ModulinoPixels leds;
// กำหนดค่าความสว่าง (ช่วง 0-255)
// 25 คือสว่างระดับต่ำ เพื่อไม่ให้แสบตาและลดการใช้กระแส
int brightness = 25;
void setup(){
    // เริ่มต้นระบบ Modulino (เตรียมการสื่อสาร I2C/Qwiic)
    Modulino.begin();
    // เริ่มต้นโมดูล LED
    leds.begin();
}

void loop(){
    // วนลูปตั้งแต่ LED ดวงที่ 0 ถึง 7 (รวมทั้งหมด 8 ดวง)
    for (int i = 0; i < 8; i++) {
        // ตั้งค่า LED ดวงที่ i ให้เป็นสีน้ำเงิน
        // และกำหนดความสว่างตามตัวแปร brightness
        leds.set(i, BLUE, brightness);
        // สั่งให้แสดงผลจริงกับ LED
        // (ถ้าไม่เรียก show() การตั้งค่าจะยังไม่แสดง)
        leds.show();
    }
}
```

```
// เรียกใช้ไลบรารีสำหรับควบคุมอุปกรณ์ Modulino
#include <Arduino_Modulino.h>
// สร้างอ็อบเจกต์สำหรับควบคุม Modulino Pixels (LED 8 ดวง)
ModulinoPixels leds;
// กำหนดค่าความสว่างสูงสุด (0-255)
int brightness = 255;
// กำหนดค่าความมืด (ปิดไฟ)
int darkness = 0;

void setup(){
  // เริ่มต้นระบบ Modulino (เตรียมการสื่อสาร I2C)
  Modulino.begin();
  // เริ่มต้นโมดูล LED
  leds.begin();
}

void loop(){
  // วนลูปตั้งแต่ LED ดวงที่ 0 ถึงดวงที่ 7 (ทั้งหมด 8 ดวง)
  for (int i = 0; i < 8; i++) {
    // ===== สีน้ำเงิน =====
    leds.set(i, BLUE, brightness); // เปิด LED ดวงที่ i เป็นสีน้ำเงิน ความสว่างสูงสุด
    leds.show();                  // แสดงผลจริง
    delay(500);                   // หน่วงเวลา 500 ms

    leds.set(i, BLUE, darkness);   // ปิด LED ดวงที่ i
    leds.show();
    delay(500);

    // ===== สีแดง =====
    leds.set(i, RED, brightness);
    leds.show();
    delay(500);

    leds.set(i, RED, darkness);
    leds.show();
    delay(500);
  }
}
```

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

โมดูล LED (Modulino Pixels) ค่าความสว่างมักจะใช้การเก็บข้อมูลขนาด 8 บิต ซึ่งทำให้สามารถไล่ระดับความสว่างได้ตั้งแต่ 0 (ปิดไฟ) ไปจนถึง 255 (สว่างสูงสุด)

```
// ===== สีเขียว =====
leds.set(i, GREEN, brightness);
leds.show();
delay(500);
```

```
leds.set(i, GREEN, darkness);
leds.show();
delay(500);
```

```
// ===== สีม่วง =====
leds.set(i, VIOLET, brightness);
leds.show();
delay(500);
```

```
leds.set(i, VIOLET, darkness);
leds.show();
delay(500);
```

```
// ===== สีขาว =====
leds.set(i, WHITE, brightness);
leds.show();
delay(500);
```

```
leds.set(i, WHITE, darkness);
leds.show();
delay(500);
```

```
}
}
```



```
// เรียกใช้ไลบรารีสำหรับควบคุม Modulino
#include "Arduino_Modulino.h"
// สร้างอ็อบเจกต์ควบคุม Modulino Pixels (LED 8 ดวง)
ModulinoPixels leds;
// กำหนดจำนวน LED ทั้งหมด
const int NUM_LEDS = 8;

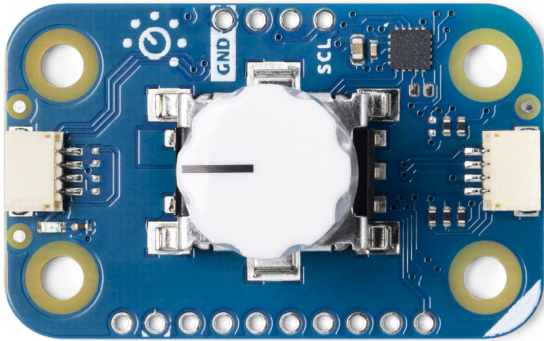
void setup() {
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้นโมดูล LED
    leds.begin();
}
// =====
// LOOP หลัก
// =====
void loop() {
    // เล่นเอฟเฟกต์สายรุ้ง 50 รอบ
    for (int i = 0; i < 50; i++) {
        rainbowCycle();
        delay(100);
    }
    // เล่นเอฟเฟกต์หายใจ 100 รอบ
    for (int i = 0; i < 100; i++) {
        breathingEffect();
        delay(30);
    }
    // เล่นเอฟเฟกต์ไฟวิ่ง 30 รอบ
    for (int i = 0; i < 30; i++) {
        chaseEffect();
        delay(80);
    }
}
```

▼ Sketch Libraries

Arduino_Modulino 0.7.0

```
// =====
// เอฟเฟกต์ที่ 1: Rainbow Cycle
// =====
void rainbowCycle() {
    // ตัวแปร static จะจำค่าระหว่างรอบ loop
    static int offset = 0;
    // วนลูปทุก LED
    for (int i = 0; i < NUM_LEDS; i++) {
        // คำนวณตำแหน่งสีแบบเลื่อนหมุน
        int colorIndex = (i + offset) % NUM_LEDS;
        // เนื่องจากไลบรารีรองรับแค่สีสำเร็จรูป
        // จึง map index ให้ใกล้เคียงสีรุ้ง
        switch(colorIndex) {
            case 0:
            case 7:
                leds.set(i, RED, 50);
                break;
            case 1:
            case 2:
                leds.set(i, GREEN, 50);
                break;
            case 3:
            case 4:
                leds.set(i, BLUE, 50);
                break;
            case 5:
            case 6:
                leds.set(i, VIOLET, 50);
                break;
        }
    }
    // แสดงผลจริง
    leds.show();
    // เลื่อนตำแหน่งสีในรอบถัดไป
    offset = (offset + 1) % NUM_LEDS;
}
```

```
// =====
// เอฟเฟกต์ที่ 2: Breathing Effect
// =====
void breathingEffect() {
    // brightness และ direction จำค่าระหว่างรอบ
    static int brightness = 0;
    static int direction = 1;
    // เพิ่มหรือลดความสว่างทีละ 5
    brightness += direction * 5;
    // ถ้าสว่างสุดหรือมืดสุด ให้กลับทิศ
    if (brightness >= 100 || brightness <= 0) {
        direction = -direction;
    }
    // ตั้งค่า LED ทุกดวงเป็นสีน้ำเงิน
    // แต่ปรับระดับความสว่างตาม brightness
    for (int i = 0; i < NUM_LEDS; i++) {
        leds.set(i, BLUE, brightness);
    }
    // แสดงผลจริง
    leds.show();
}
// =====
// เอฟเฟกต์ที่ 3: Chase Effect (ไฟวิ่ง)
// =====
void chaseEffect() {
    static int position = 0;
    // ปิดไฟทุกดวงก่อน
    for (int i = 0; i < NUM_LEDS; i++) {
        leds.set(i, WHITE, 0);
    }
    // เปิดไฟดวงปัจจุบันสว่างสุด
    leds.set(position, WHITE, 100);
    // ทำหางไฟ (ความสว่างต่ำกว่า)
    if (position > 0) {
        leds.set(position - 1, WHITE, 30);
    } else {
        leds.set(NUM_LEDS - 1, WHITE, 30);
    }
    // แสดงผลจริง
    leds.show();
    // เลื่อนตำแหน่งไปดวงถัดไป
    position = (position + 1) % NUM_LEDS;
}
```



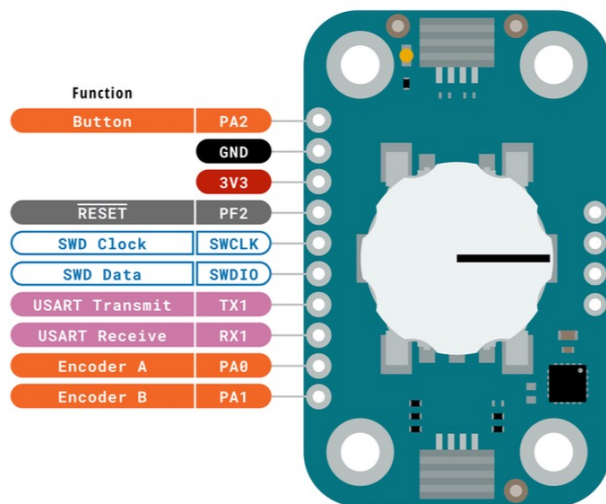
Modulino® **KNOB**

Modulino® Knob เป็นโมดูลอินพุตแบบหมุนอัจฉริยะที่ให้ความแม่นยำสูง ออกแบบมาเพื่อใช้ในการควบคุมค่าต่าง ๆ ในโปรเจกต์อย่างละเอียด

Modulino® Knob มาพร้อมกับเอ็นโค้ดเดอร์แบบหมุนชนิด **Quadrature Rotary Encoder (รุ่น PEC11J-9215F-S0015)** ทำงานร่วมกับไมโครคอนโทรลเลอร์ **STM32C011F4** บนบอร์ด ช่วยให้สามารถรับค่าการหมุนได้อย่างแม่นยำ พร้อมรองรับการกดปุ่มในตัว โมดูลนี้สื่อสารผ่าน I²C (อินเทอร์เฟซ Qwiic) เป็นค่าเริ่มต้น แต่ยังสามารถโปรแกรมใหม่ (Reprogram) หรือเข้าถึงผ่านอินเทอร์เฟซการสื่อสารอื่น ๆ ได้ สำหรับโปรเจกต์ขั้นสูง

ค่าเริ่มต้น (Default) คือ **0x76**

เหมาะอย่างยิ่งสำหรับ:



- การสร้างระบบควบคุมระดับเสียงหรือความสว่าง
- การเลือกเมนู (Menu selection) บนหน้าจอแสดงผล
- โปรเจกต์ที่ต้องการการปรับแต่งค่าตัวเลขอย่างละเอียดและรวดเร็ว

ฟังก์ชันหลัก (Key Functions)

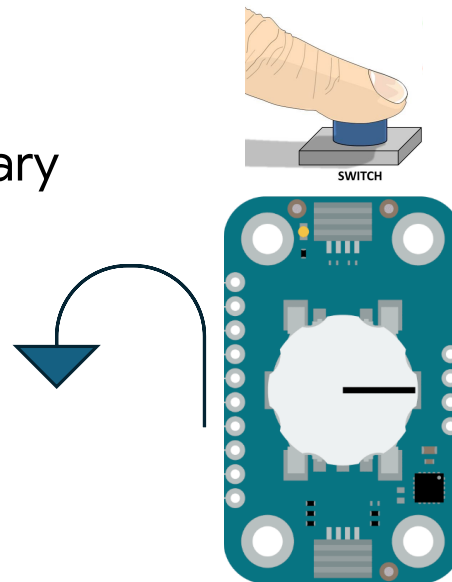
- **get():**
คืนค่าตัวเลขที่สัมพันธ์กับการหมุนของปุ่มหมุน (Knob) โดยค่าจะเพิ่มหรือลดตามทิศทางและจำนวนครั้งที่หมุน
- **isPressed():**
ตรวจสอบสถานะของปุ่มกดที่อยู่ในตัว Knob (คืนค่า true เมื่อกด และ false เมื่อไม่กด)
- **set():**
กำหนดค่าตำแหน่งเริ่มต้นใหม่ของตัวเข้ารหัสแบบหมุน (Encoder) ใช้สำหรับรีเซ็ตหรือกำหนดจุดเริ่มต้นของค่าการหมุน

knob.get() ของ Modulino Knob (Rotary Encoder)

ทำงานแบบ incremental encoder

หมายความว่า:

- หมุนตามเข็มนาฬิกา → ค่าเพิ่มขึ้นเรื่อย ๆ
- หมุนทวนเข็มนาฬิกา → ค่าลดลงเรื่อย ๆ
- ไม่มีจุดสิ้นสุดเหมือน potentiometer



ค่าภายในเป็นชนิด int บน UNO Q (32-bit system) → int มีช่วงประมาณ:

-2,147,483,648 ถึง 2,147,483,647

ซึ่งในทางปฏิบัติ หมุนไม่ถึงแน่นอน 😊

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

📖 MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับควบคุมอุปกรณ์ Modulino
#include <Arduino_Modulino.h>
// เรียกใช้ไลบรารี RouterBridge (ใช้กับ UNO Q สำหรับ Monitor.print)
#include <Arduino_RouterBridge.h>
// สร้างอ็อบเจกต์สำหรับควบคุม Modulino Knob (Rotary Encoder)
ModulinoKnob knob;

void setup() {
    // เริ่มต้นการสื่อสารกับ Monitor (แสดงผลบนคอมพิวเตอร์)
    Monitor.begin();
    // เริ่มต้นระบบ Modulino (เตรียมการสื่อสาร I2C)
    Modulino.begin();
    // เริ่มต้นโมดูล Knob
    knob.begin();
}

void loop(){
    // อ่านค่าตำแหน่งปัจจุบันของ Knob
    // ค่าจะเพิ่มหรือลดตามการหมุน
    int position = knob.get();
    // ตรวจสอบว่าปุ่มกดบน Knob ถูกกดอยู่หรือไม่
    bool click = knob.isPressed();
    // แสดงข้อความบอกตำแหน่งปัจจุบัน
    Monitor.print("ตำแหน่งปัจจุบันคือ : ");
    Monitor.println(position);
    // ถ้ามีการกดปุ่มบน Knob
    if(click){
        Monitor.println("มีการกดคลิก!");
    }
}
```

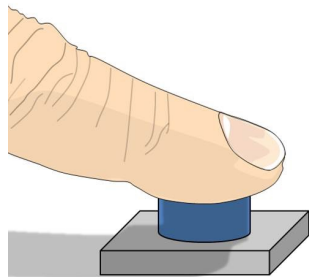

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

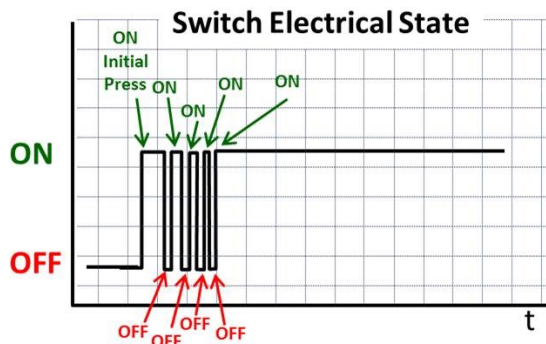
📖 MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// เรียกใช้ RouterBridge สำหรับใช้ Monitor แสดงผล
#include <Arduino_RouterBridge.h>
// สร้างอ็อบเจกต์สำหรับควบคุม Knob (Rotary Encoder)
ModulinoKnob knob;
// ===== การตั้งค่าเมนู =====
// จำนวนรายการเมนูทั้งหมด
const int MENU_ITEMS = 5;
// สร้างอาร์เรย์เก็บชื่อเมนู
String menuOptions[] = {"ตั้งค่า", "หน้าจอ", "เสียง", "เครือข่าย", "ออก"};
// เก็บตำแหน่งเมนูที่กำลังถูกเลือก
int currentSelection = 0;
// เก็บค่าตำแหน่งก่อนหน้าของ Knob
int lastPosition = 0;
// เก็บสถานะปุ่มก่อนหน้า (ใช้ตรวจจับการกดแบบปล่อยปุ่ม)
bool lastButtonState = false;

void setup() {
    // เริ่มการสื่อสารกับ Monitor
    Monitor.begin();
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้น Knob
    knob.begin();
    // กำหนดค่าเริ่มต้นของ encoder เป็น 0
    knob.set(0);
    lastPosition = 0;
    // แสดงเมนูครั้งแรก
    displayMenu();
}
```



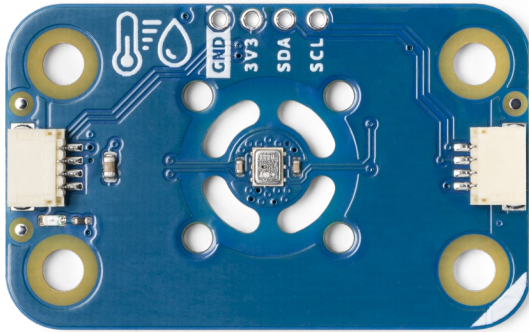
SWITCH



```
// ===== วงจรหลัก =====
void loop() {
    // อ่านค่าตำแหน่งปัจจุบันของ Knob
    int currentPosition = knob.get();
    // อ่านสถานะปุ่ม (true = กดอยู่)
    bool buttonPressed = knob.isPressed();
    // คำนวณว่าหมุนไปกี่สเกลป์จากรอบก่อน
    int rotation = currentPosition - lastPosition;
    // ===== ตรวจจับการหมุน =====
    if (rotation != 0) {
        // ถ้าหมุนตามเข็มนาฬิกา (ค่ามากขึ้น)
        if (rotation > 0) {
            currentSelection++;
            // ถ้าเกินจำนวนเมนู → กลับไปข้อแรก (วนลูป)
            if (currentSelection >= MENU_ITEMS) {
                currentSelection = 0;
            }
        } else { // ถ้าหมุนทวนเข็มนาฬิกา
            currentSelection--;
            // ถ้าต่ำกว่า 0 → ไปข้อสุดท้าย
            if (currentSelection < 0) {
                currentSelection = MENU_ITEMS - 1;
            }
        }
        // อัปเดตตำแหน่งล่าสุด
        lastPosition = currentPosition;
        // แสดงเมนูใหม่
        displayMenu();
    }
    // ===== ตรวจจับการกดปุ่ม =====
    // ตรวจสอบ "ปล่อยปุ่ม" เพื่อป้องกันการเรียกซ้ำหลายครั้ง
    if (!buttonPressed && lastButtonState) {
        selectMenuItem();
    }
    // อัปเดตสถานะปุ่มล่าสุด
    lastButtonState = buttonPressed;
    // หน่วงเล็กน้อยช่วยลดอาการสั่นของสัญญาณ (debounce)
    delay(50);
}
```

```
// ===== ฟังก์ชันแสดงเมนู =====
void displayMenu() {
    Monitor.println("\n===== เมนู =====");
    // วนลูปแสดงรายการเมนูทั้งหมด
    for (int i = 0; i < MENU_ITEMS; i++) {
        // ถ้ารายการนี้คือรายการที่ถูกเลือก
        if (i == currentSelection) {
            Monitor.print("> "); // แสดงเครื่องหมาย >
        } else {
            Monitor.print(" "); // เว้นวรรคปกติ
        }
        // แสดงชื่อเมนู
        Monitor.println(menuOptions[i]);
    }
    Monitor.println("=====");
    Monitor.println("หมุนเพื่อเลือกเมนู, กดเพื่อตกลง");
}

// ===== ฟังก์ชันเมื่อเลือกเมนู =====
void selectMenuItem() {
    Monitor.print("\n✓ เลือกแล้ว: ");
    Monitor.println(menuOptions[currentSelection]);
    // ตรวจสอบว่าผู้ใช้เลือกเมนูข้อไหน
    switch(currentSelection) {
        case 0:
            Monitor.println("กำลังเปิดการตั้งค่า...");
            break;
        case 1:
            Monitor.println("กำลังเปิดการตั้งค่าหน้าจอ...");
            break;
        case 2:
            Monitor.println("กำลังเปิดเพื่อจัดการระบบเสียง...");
            break;
        case 3:
            Monitor.println("กำลังเปิดเพื่อตั้งค่าเครือข่าย...");
            break;
        case 4:
            Monitor.println("ออกจากเมนู...");
            break;
    }
    // หน่วงเวลาเพื่อให้ผู้ใช้เห็นข้อความ
    delay(1000);
    // กลับมาแสดงเมนูอีกครั้ง
    displayMenu();
}
```



Modulino® **THERMO**

Modulino® Thermo คือโมดูลเซนเซอร์อัจฉริยะในซีรีส์ Modulino® ที่ออกแบบมาเพื่อการตรวจวัดสภาพแวดล้อมอย่างแม่นยำ

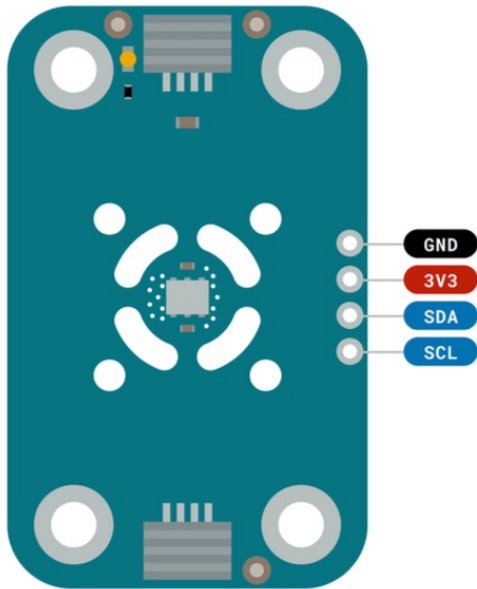
Modulino® Thermo ซึ่งมาพร้อมเซนเซอร์ **HS3003** สามารถวัดได้ทั้งอุณหภูมิและความชื้น เหมาะอย่างยิ่งสำหรับงานตรวจวัดสภาพแวดล้อม การควบคุมสภาพอากาศภายในอาคาร และการใช้งานด้าน IoT หลากหลายประเภท

ให้ค่าการวัดที่แม่นยำในขนาดโมดูลที่กะทัดรัด และสามารถผสานการทำงานเข้ากับระบบนิเวศ Qwiic ได้อย่างราบรื่น

ค่าเริ่มต้น (Default) คือ **0x44**

เหมาะอย่างยิ่งสำหรับ:

- งานตรวจวัดสภาพแวดล้อม (Environmental monitoring)
- การควบคุมสภาพอากาศภายในอาคาร (Indoor climate control)
- โปรเจกต์ IoT หลากหลายประเภทที่ต้องการข้อมูลสภาพอากาศที่เชื่อถือได้



▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

📖 MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include <Arduino_Modulino.h>
// ไลบรารีสำหรับใช้ Monitor แสดงผลผ่านคอมพิวเตอร์
#include <Arduino_RouterBridge.h>
// สร้างอ็อบเจกต์สำหรับใช้งาน โมดูลวัดอุณหภูมิและความชื้น
ModulinoThermo thermo;
// ----- ตัวแปร Global -----
// เก็บค่าอุณหภูมิ (หน่วยองศาเซลเซียส)
float celsius = 0;
// เก็บค่าความชื้นสัมพัทธ์ (หน่วยเปอร์เซ็นต์)
float humidity = 0;
// ตัวแปรเก็บข้อความทั้งหมดที่จะนำไปแสดงผล
String msg;

void setup(){
    // เริ่มต้นการสื่อสารกับ Monitor
    Monitor.begin();
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้นการทำงานของโมดูล Thermo
    thermo.begin();
}
```

ฟังก์ชันหลัก (Key Functions)

- **getTemperature():**
ส่งคืนค่าการวัดอุณหภูมิในหน่วยองศาเซลเซียส (°C)
- **getHumidity():**
ส่งคืนค่าความชื้นสัมพัทธ์ในหน่วยเปอร์เซ็นต์ (%)
- **Modulino.begin():**
ใช้เริ่มต้นการทำงานของระบบ Modulino (โดยค่าเริ่มต้นจะการใช้การสื่อสารผ่าน Wire1)

```
void loop(){
    // อ่านค่าอุณหภูมิจากเซนเซอร์
    celsius = thermo.getTemperature();
    // อ่านค่าความชื้นจากเซนเซอร์
    humidity = thermo.getHumidity();
    // ----- สร้างข้อความสำหรับแสดงผล -----
    // เส้นค้นข้อความ
    String msg_0 = "\r\n-----";
    // ข้อความอธิบายอุณหภูมิ
    String msg_1 = "\r\nอุณหภูมิมีค่าเท่ากับ (องศาเซลเซียส) :";
    // แปลงค่าอุณหภูมิ (float) เป็น String
    String msg_2 = String(celsius);
    // ข้อความอธิบายความชื้น
    String msg_3 = "\r\nความชื้นมีค่าเท่ากับ (เปอร์เซ็นต์) :";
    // แปลงค่าความชื้น (float) เป็น String
    String msg_4 = String(humidity);
    // รวมข้อความทั้งหมดเข้าด้วยกัน
    msg = msg_0 + msg_1 + msg_2 + msg_3 + msg_4 + msg_0;
    // แสดงผลข้อความผ่าน Monitor
    Monitor.print(msg);
    // หน่วงเวลา 1 วินาที ก่อนอ่านค่าใหม่
    delay(1000);
}
```


▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

📖 MsgPack 0.4.2

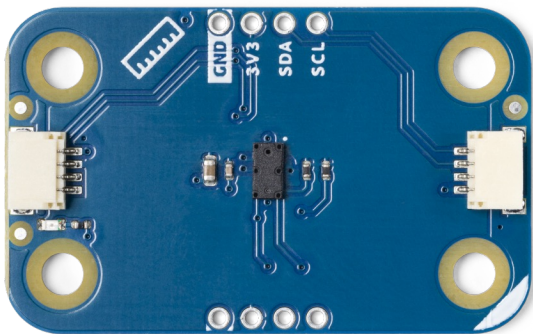
```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// ไลบรารีสำหรับแสดงผลผ่าน Monitor
#include <Arduino_RouterBridge.h>
// สร้างอ็อบเจกต์สำหรับเซนเซอร์วัดอุณหภูมิและความชื้น
ModulinoThermo thermo;
// กำหนดค่าเกณฑ์อุณหภูมิ (แจ้งเตือนเมื่อมากกว่า 30°C)
const float TEMP_THRESHOLD = 30.0;
// กำหนดค่าเกณฑ์ความชื้น (แจ้งเตือนเมื่อมากกว่า 70%)
const float HUMIDITY_THRESHOLD = 70.0;
// ตัวแปรเก็บข้อความแสดงผล
String msg, msg_t, msg_h;

void setup(){
    // เริ่มต้นการสื่อสารกับ Monitor
    Monitor.begin();
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้นการทำงานของเซนเซอร์ Thermo
    thermo.begin();
    // แสดงข้อความเมื่อระบบเริ่มทำงาน
    Monitor.println("ระบบตรวจวัดสภาพแวดล้อมเริ่มทำงานแล้ว");
}
```

```
void loop(){
    // อ่านค่าอุณหภูมิจากเซนเซอร์ (หน่วย °C)
    float temp = thermo.getTemperature();
    // อ่านค่าความชื้นสัมพัทธ์ (หน่วย %)
    float hum = thermo.getHumidity();
    // ----- ตรวจสอบอุณหภูมิ -----
    // ถ้าอุณหภูมิสูงกว่าเกณฑ์ที่กำหนด
    if(temp > TEMP_THRESHOLD) {
        String msg_1 = "⚠ อุณหภูมิสูง: ";
        String msg_2 = String(temp, 2); // แสดงทศนิยม 2 ตำแหน่ง
        String msg_3 = "°C";
        // รวมข้อความแจ้งเตือน
        msg_t = msg_1 + msg_2 + msg_3;
        // แสดงผลแจ้งเตือน
        Monitor.println(msg_t);
    }

    // ----- ตรวจสอบความชื้น -----
    // ถ้าความชื้นสูงกว่าเกณฑ์ที่กำหนด
    if(hum > HUMIDITY_THRESHOLD) {
        String msg_1 = "⚠ ความชื้นในอากาศสูง: ";
        String msg_2 = String(hum, 2); // แสดงทศนิยม 2 ตำแหน่ง
        String msg_3 = "%";
        // รวมข้อความแจ้งเตือน
        msg_h = msg_1 + msg_2 + msg_3;
        // แสดงผลแจ้งเตือน
        Monitor.println(msg_h);
    }

    // ----- กรณีค่าปกติ -----
    // ถ้าอุณหภูมิและความชื้นไม่เกินค่าที่กำหนด
    if(temp <= TEMP_THRESHOLD && hum <= HUMIDITY_THRESHOLD) {
        String msg_1 = "อุณหภูมิ: ";
        String msg_2 = String(temp, 2);
        String msg_3 = "°C, ความชื้น: ";
        String msg_4 = String(hum, 2);
        String msg_5 = "%";
        // รวมข้อความแสดงค่าปกติ
        msg = msg_1 + msg_2 + msg_3 + msg_4 + msg_5;
        // แสดงค่าปกติ
        Monitor.println(msg);
    }
    // หน่วงเวลา 5 วินาที ก่อนอ่านค่ารอบถัดไป
    delay(5000);
}
```



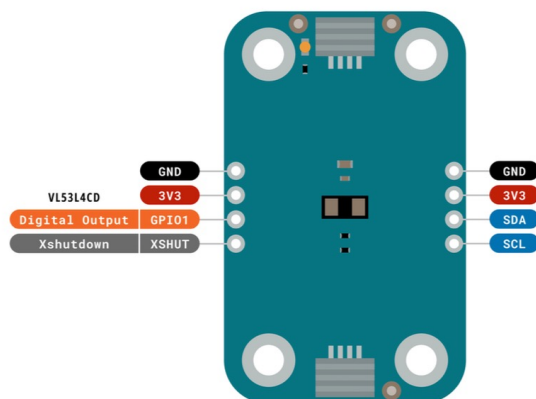
Modulino® DISTANCE

Modulino® Distance เป็นโมดูลเซนเซอร์วัดระยะทางอัจฉริยะในซีรีส์ Modulino® ที่ออกแบบมาให้มีขนาดกะทัดรัดและใช้งานง่าย

Modulino® Distance ซึ่งมาพร้อมเซนเซอร์วัดระยะทางแบบ **Time-of-Flight** รุ่น **VL53L4CDV0DH/1** สามารถวัดระยะทางได้อย่างแม่นยำในรูปแบบโมดูลขนาดกะทัดรัดและใช้งานง่าย

เหมาะสำหรับงานตรวจจับวัตถุใกล้เคียง (Proximity Detection) ระบบหลบหลีกสิ่งกีดขวาง (Obstacle Avoidance) และการประยุกต์ใช้งานด้านระบบตรวจจับอัจฉริยะหลากหลายรูปแบบ

ค่าเริ่มต้น (Default) คือ 0x29



เหมาะอย่างยิ่งสำหรับ:

- ระบบหลบหลีกสิ่งกีดขวาง (Obstacle Avoidance) ในหุ่นยนต์
- การตรวจจับวัตถุใกล้เคียง (Proximity Detection)
- ระบบตรวจจับอัจฉริยะหลากหลายรูปแบบ ที่ต้องการการวัดระยะที่เชื่อถือได้

ฟังก์ชันหลัก (Key Functions)

- **available():**
ตรวจสอบว่ามีข้อมูลระยะทางใหม่พร้อมให้อ่านหรือไม่
- **get():**
ดึงค่าระยะทางที่วัดได้จากเซนเซอร์ (ค่าเริ่มต้นเป็นหน่วยเซนติเมตร cm)

▼ Sketch Libraries

Arduino_Modulino 0.7.0
MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// ไลบรารีสำหรับแสดงผลผ่าน Monitor
#include <Arduino_RouterBridge.h>
// ----- สร้างอ็อบเจกต์ -----
// สร้างอ็อบเจกต์สำหรับเซนเซอร์วัดระยะทาง
ModulinoDistance distance;
// ตัวแปรสำหรับเก็บข้อความที่จะแสดงผล
String msg;

void setup() {
    // เริ่มต้นการสื่อสารกับ Monitor
    Monitor.begin();
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้นการทำงานของเซนเซอร์วัดระยะทาง
    distance.begin();
}
```

```
void loop() {
    // ตรวจสอบว่ามีข้อมูลระยะทางใหม่พร้อมให้อ่านหรือไม่
    if (distance.available()) {
        // อ่านค่าระยะทางจากเซนเซอร์
        int measure = distance.get();
        // สร้างข้อความสำหรับแสดงผล
        String msg_1 = "ค่าระยะทาง :";
        String msg_2 = String(measure);
        // รวมข้อความเข้าด้วยกัน
        msg = msg_1 + msg_2;
        // แสดงค่าระยะทางผ่าน Monitor
        Monitor.println(msg);
    }
    // หน่วงเวลาเล็กน้อยเพื่อลดความถี่ในการอ่านค่า
    delay(10);
}
```

▼ Sketch Libraries

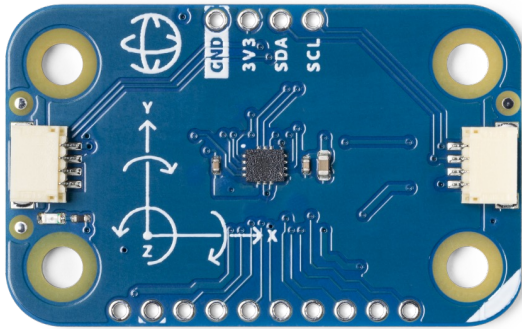
Arduino_Modulino 0.7.0

MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// ไลบรารีสำหรับแสดงผลผ่าน Monitor
#include <Arduino_RouterBridge.h>
// ----- สร้างอ็อบเจกต์ -----
// สร้างอ็อบเจกต์สำหรับเซนเซอร์วัดระยะทาง
ModulinoDistance distance;
// ----- กำหนดช่วงระยะ (หน่วยเซนติเมตร) ---
// ระยะอันตรายมาก (น้อยกว่า 10 ซม.)
const int DANGER_ZONE = 10;
// ระยะเตือน (10-30 ซม.)
const int WARNING_ZONE = 30;
// ระยะควรระวัง (30-50 ซม.)
const int CAUTION_ZONE = 50;
// ระยะปลอดภัย (50-100 ซม.)
const int SAFE_ZONE = 100;
// ตัวแปรเก็บข้อความที่จะแสดงผล
String msg;

void setup() {
    // เริ่มต้นการสื่อสารกับ Monitor
    Monitor.begin();
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้นการทำงานของเซนเซอร์วัดระยะทาง
    distance.begin();
}
```

```
void loop() {
    // ตรวจสอบว่ามีข้อมูลระยะทางใหม่หรือไม่
    if (distance.available()) {
        // อ่านค่าระยะทางจากเซนเซอร์ (หน่วยเซนติเมตร)
        int measure = distance.get();
        // ----- ตรวจสอบช่วงระยะ -----
        // ถ้าระยะน้อยกว่า 10 ซม. (อันตรายมาก)
        if (measure < DANGER_ZONE) {
            String msg_1 = "หยุด! ระยะทาง: ";
            String msg_2 = String(measure);
            String msg_3 = " เซนติเมตร - ใกล้แบบนี้ ชนแน่นอน!";
            // รวมข้อความ
            msg = msg_1 + msg_2 + msg_3;
            // แสดงผล
            Monitor.println(msg);
        }
        // ถ้าระยะอยู่ระหว่าง 10-30 ซม. (เตือน)
        else if (measure < WARNING_ZONE) {
            String msg_1 = "เตือน - ระยะทาง: ";
            String msg_2 = String(measure);
            String msg_3 = " เซนติเมตร - ใกล้ตัวมาก";
            msg = msg_1 + msg_2 + msg_3;
            Monitor.println(msg);
        }
        // ถ้าระยะอยู่ระหว่าง 30-50 ซม. (ควรระวัง)
        else if (measure < CAUTION_ZONE) {
            String msg_1 = "ระวัง - ระยะทาง: ";
            String msg_2 = String(measure);
            String msg_3 = " เซนติเมตร - มีวัตถุขวางทาง";
            msg = msg_1 + msg_2 + msg_3;
            Monitor.println(msg);
        }
        // ถ้าระยะอยู่ระหว่าง 50-100 ซม. (ปลอดภัย)
        else if (measure < SAFE_ZONE) {
            String msg_1 = "ระยะปลอดภัย - ระยะทาง: ";
            String msg_2 = String(measure);
            String msg_3 = " เซนติเมตร - ทางโล่ง";
            msg = msg_1 + msg_2 + msg_3;
            Monitor.println(msg);
        }
        // ถ้ามากกว่า 100 ซม. (ไม่มีสิ่งกีดขวางใกล้เคียง)
        else {
            String msg_1 = "✓ ไม่มีสิ่งกีดขวาง - ระยะทาง: ";
            String msg_2 = String(measure);
            String msg_3 = " เซนติเมตร";
            msg = msg_1 + msg_2 + msg_3;
            Monitor.println(msg);
        }
    }
    // หน่วงเวลา 100 มิลลิวินาที (อัปเดตประมาณ 10 ครั้งต่อวินาที)
    delay(100);
}
```

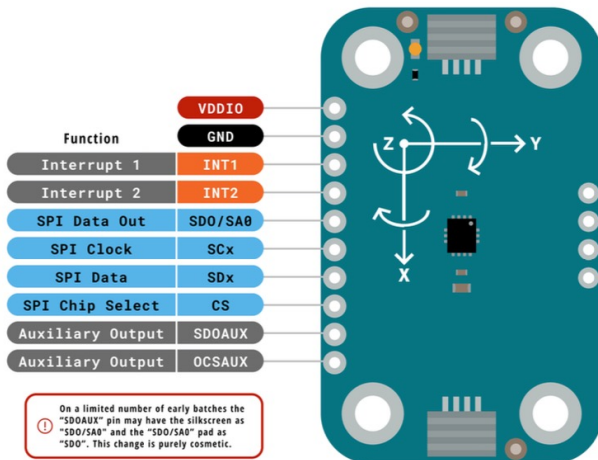


Modulino® Movement เป็นโมดูลเซนเซอร์อัจฉริยะในซีรีส์ Modulino® ที่ถูกออกแบบมาเพื่อการตรวจจับการเคลื่อนไหวและการสั่นสะเทือนที่แม่นยำสูง

Modulino® Movement ซึ่งมาพร้อมเซนเซอร์ LSM6DSOXTR สามารถวัดทั้งความเร่ง (Acceleration) และการหมุน (Rotation) ให้ข้อมูลครบถ้วนสำหรับงานด้านการตรวจจับการเคลื่อนไหว เหมาะสำหรับโปรเจกต์ที่เกี่ยวข้องกับการจดจำท่าทาง (Gesture Recognition) เครื่องนับก้าว (Pedometer) และการตรวจวัดแรงสั่นสะเทือน (Vibration Monitoring)

Modulino® **MOVEMENT**

ค่าเริ่มต้น (Default) คือ 0x6A



เหมาะอย่างยิ่งสำหรับ:

- การจดจำท่าทาง (Gesture Recognition)
- เครื่องนับก้าว (Pedometer)
- การตรวจวัดแรงสั่นสะเทือน (Vibration Monitoring)



วิธีเปลี่ยนค่า I²C Address ของเซนเซอร์ **Modulino[®] Movement**

👉 ให้ ตัดลายวงจร (jumper) ตำแหน่ง 0x6A เพื่อยกเลิกค่า Address เดิม (0x6A)

👉 ให้ บัดกรีเชื่อมลายวงจร (short) ตำแหน่ง 0x6B เพื่อกำหนด Address ใหม่เป็น 0x6B

🧠 ทำไปเพื่ออะไร?

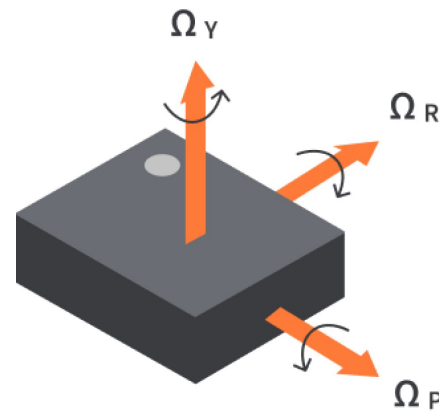
ใช้ในกรณีที่: มีเซนเซอร์ **Modulino[®] Movement** ตัวเดียวกันมากกว่า 1 ตัวบนบัส I²C ต้องการไม่ให้ Address ซ้ำกัน

หมายเหตุ: ห้ามให้จุด 0x6A และ 0x6B ถูกเชื่อมพร้อมกัน เพราะจะทำให้ไฟฟ้าลัดวงจร เสี่ยงทำให้บอร์ดหรือเซนเซอร์พังได้

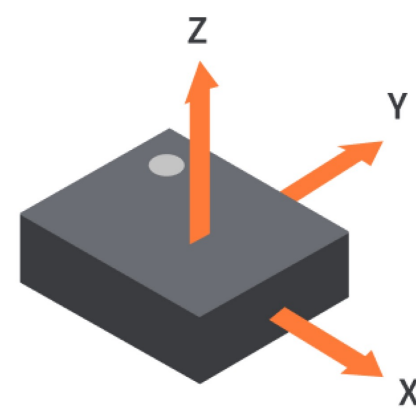
✅ ควรเลือกบัดกรีแค่จัมเปอร์เดียวเท่านั้น และควรถอดไฟออกก่อนทำการตัดหรือบัดกรีทุกครั้ง เพื่อความปลอดภัย

Key Functions (ฟังก์ชันหลัก)

- `update()` : อัปเดตค่าการอ่านข้อมูลจากเซนเซอร์ล่าสุด
- `getX()`, `getY()`, `getZ()` : ดึงค่าความเร่ง (Acceleration) ในหน่วย g ของแกน X, Y และ Z
- `getRoll()`, `getPitch()`, `getYaw()` : ดึงค่าการหมุนเชิงมุม (Angular rotation) ในหน่วยองศาต่อวินาที (dps) ของแกน Roll, Pitch และ Yaw



Gyroscope sensing
Angular orientation



Accelerometer sensing
axis orientation

ทำไมต้องปรับเทียบ (Calibrate)?

เซนเซอร์วัดการเคลื่อนไหว (เช่น accelerometer / gyroscope) แม้จะวางนิ่ง ๆ ก็อาจอ่านค่าได้แบบนี้ เช่น:

X = 0.12
Y = -0.08
Z = 9.65

แทนที่จะเป็น

X = 0
Y = 0
Z = 9.81

Z ≈ 9.81 ตอนวางนิ่ง

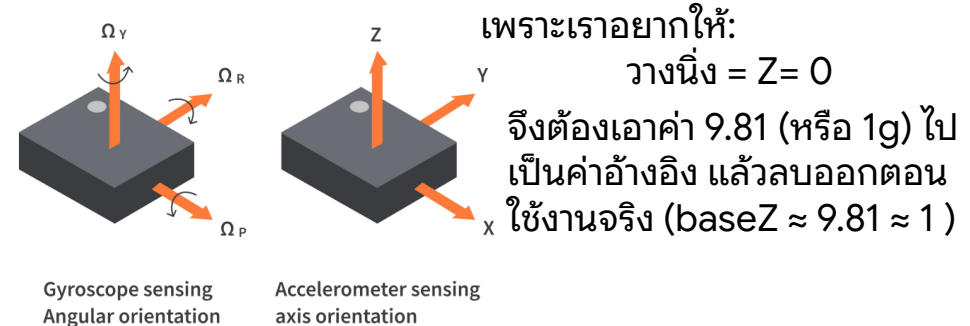
เพราะว่า Accelerometer วัดความเร่ง (Acceleration) และ “แรงโน้มถ่วงของโลก” ก็เป็นความเร่งชนิดหนึ่ง ค่าความเร่งโน้มถ่วงของโลก $\approx 9.81 \text{ m/s}^2 \approx 1 \text{ m/s}^2$

สาเหตุเพราะ:

- ค่าคลาดเคลื่อนจากโรงงาน (sensor bias)
- อุณหภูมิ
- การเอียงเล็กน้อย
- สัญญาณรบกวน

ถ้าไม่ปรับเทียบ เวลาตรวจจับการเคลื่อนไหวจะผิดพลาด เช่น:

- วางนิ่งแต่ระบบคิดว่าขยับ
- ค่ามุม Roll/Pitch/Yaw เพี้ยนตลอดเวลา



📌 ถ้าไม่มี `calibrateSensor` จะเกิดอะไรขึ้น?

- ค่าเริ่มต้นจะไม่เป็นศูนย์
- ตรวจจับ motion ผิดพลาดง่าย
- มุมเอียงค่อย ๆ ลอย (drift)
- ระบบควบคุมจะไม่แม่นยำ

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

📖 MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// ไลบรารีสำหรับแสดงผลผ่าน Monitor
#include <Arduino_RouterBridge.h>
// ----- สร้างอ็อบเจกต์ -----
// สร้างอ็อบเจกต์สำหรับโมดูลตรวจจับการเคลื่อนไหว (Accelerometer + Gyroscope)
ModulinoMovement movement;
// ตัวแปรเก็บค่า Acceleration (ความเร่ง) แกน X, Y, Z
float x, y, z;
// ตัวแปรเก็บค่า Gyroscope (การหมุน) roll, pitch, yaw
float roll, pitch, yaw;
// ตัวแปรสำหรับเก็บข้อความที่จะแสดงผล
String msg;

void setup() {
    // เริ่มต้นการสื่อสารกับ Monitor
    Monitor.begin();
    // เริ่มต้นระบบ Modulino (I2C Communication)
    Modulino.begin();
    // เริ่มต้นการทำงานของเซนเซอร์ตรวจจับการเคลื่อนไหว
    movement.begin();
}
```

```
void loop() {
    // อัปเดตข้อมูลใหม่จากเซนเซอร์
    movement.update();
    // ----- อ่านค่า Acceleration -----
    // อ่านค่าความเร่งในแต่ละแกน
    x = movement.getX();
    y = movement.getY();
    z = movement.getZ();
    // ----- อ่านค่า Gyroscope -----
    // อ่านค่าการหมุน (มุมเอียง)
    roll = movement.getRoll();
    pitch = movement.getPitch();
    yaw = movement.getYaw();
    // ----- สร้างข้อความแสดงค่า Acceleration -----
    String msg_0 = "ค่าของ Acceleration: ";
    String msg_1 = String(x, 3);    // แสดงทศนิยม 3 ตำแหน่ง
    String msg_2 = ", ";
    String msg_3 = String(y, 3);
    String msg_4 = ", ";
    String msg_5 = String(z, 3);
    String msg_6 = ", ";
    // ตัวคั่นระหว่าง Acceleration และ Gyroscope
    String msg_7 = " | ค่าของ Gyroscope: ";
    // ----- สร้างข้อความแสดงค่า Gyroscope -----
    String msg_8 = String(roll, 1);  // แสดงทศนิยม 1 ตำแหน่ง
    String msg_9 = ", ";
    String msg_10 = String(pitch, 1);
    String msg_11 = ", ";
    String msg_12 = String(yaw, 1);
    // รวมข้อความทั้งหมดเข้าด้วยกัน
    msg = msg_0 + msg_1 + msg_2 + msg_3 + msg_4 + msg_5 + msg_6
        + msg_7 + msg_8 + msg_9 + msg_10 + msg_11 + msg_12;
    // แสดงผลผ่าน Monitor
    Monitor.println(msg);
    // หน่วงเวลา 200 มิลลิวินาที (อัปเดตประมาณ 5 ครั้งต่อวินาที)
    delay(200);
}
```

▼ Sketch Libraries

Arduino_Modulino 0.7.0
MsgPack 0.4.2

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include "Arduino_Modulino.h"
// ไลบรารีสำหรับแสดงผลผ่าน Monitor
#include <Arduino_RouterBridge.h>
// ----- สร้างอ็อบเจกต์ -----
// โมดูลตรวจสอบการเคลื่อนไหว (Accelerometer + Gyroscope)
ModulinoMovement movement;
// ----- ค่ากำหนดเกณฑ์การตรวจจับ -----
// ค่าเกณฑ์ความเร่ง (หน่วย g)
const float MOTION_THRESHOLD = 0.15;
// ค่าเกณฑ์การหมุน (องศาต่อวินาที)
const float ROTATION_THRESHOLD = 20.0;
// ----- ตัวแปรสำหรับการปรับเทียบ (Calibration) -----
// ค่าเริ่มต้นของแกน X, Y, Z (คาดว่าเมื่อวางราบ Z ≈ 1g)
float baseX = 0, baseY = 0, baseZ = 1.0;
// ค่าเริ่มต้นของมุม Roll, Pitch, Yaw
float baseRoll = 0, basePitch = 0, baseYaw = 0;
// สถานะว่าปรับเทียบแล้วหรือยัง
bool isCalibrated = false;
// ----- ตัวแปรติดตามสถานะการเคลื่อนไหว -----
bool inMotion = false; // ขณะนี้กำลังเคลื่อนไหวหรือไม่
unsigned long motionStartTime = 0; // เวลาเริ่มเคลื่อนไหว
unsigned long stillStartTime = 0; // เวลาเริ่มอยู่นิ่ง
// ระยะเวลาที่ต้องนิ่ง (2 วินาที) เพื่อถือว่าหยุดเคลื่อนไหว
const unsigned long STILL_TIMEOUT = 2000;

void setup() {
    // เริ่มต้น Monitor และระบบ Modulino
    Monitor.begin();
    Modulino.begin();
    movement.begin();
    // แสดงข้อความเริ่มต้นระบบ
    Monitor.println("ระบบตรวจจับการเคลื่อนไหว");
    Monitor.println("=====");
    Monitor.println("กรุณาวางอุปกรณ์ให้นิ่งเพื่อทำการปรับเทียบ (คาลิเบรต)...");
    delay(2000); // รอ 2 วินาที
    calibrateSensor(); // เรียกฟังก์ชันปรับเทียบ
}

// ----- ลูปหลัก -----
void loop() {
    // ทำงานเฉพาะเมื่อปรับเทียบแล้ว
    if (isCalibrated) {
        detectMotion();
        delay(50); // อัปเดตทุก 50 ms (~20 ครั้งต่อวินาที)
    }
}
```

```
// ----- ฟังก์ชันปรับเทียบเซนเซอร์ -----
void calibrateSensor() {
    const int samples = 50; // จำนวนตัวอย่างที่ใช้เฉลี่ยค่า
    float sumX = 0, sumY = 0, sumZ = 0;
    float sumRoll = 0, sumPitch = 0, sumYaw = 0;
    // เก็บค่าหลายครั้งเพื่อหาค่าเฉลี่ย
    for (int i = 0; i < samples; i++) {
        movement.update();
        sumX += movement.getX();
        sumY += movement.getY();
        sumZ += movement.getZ();
        sumRoll += movement.getRoll();
        sumPitch += movement.getPitch();
        sumYaw += movement.getYaw();
        delay(20);
    }
    // คำนวณค่าเฉลี่ยเป็นค่าพื้นฐาน (Baseline)
    baseX = sumX / samples;
    baseY = sumY / samples;
    baseZ = sumZ / samples;
    baseRoll = sumRoll / samples;
    basePitch = sumPitch / samples;
    baseYaw = sumYaw / samples;
    isCalibrated = true; // ตั้งค่าว่าปรับเทียบเสร็จแล้ว
    Monitor.println("✓ การปรับเทียบ(คาลิเบรต)เสร็จสมบูรณ์แล้ว!");
    Monitor.println();
}
```

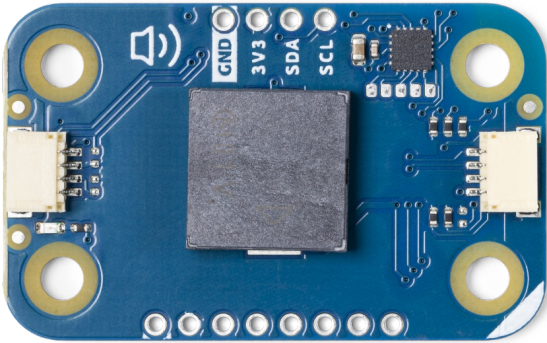


```
// ----- ฟังก์ชันตรวจจับการเคลื่อนไหว -----
void detectMotion() {
    movement.update(); // อัปเดตค่าจากเซนเซอร์
    // คำนวณค่าความแตกต่างจากค่า baseline
    float deltaX = abs(movement.getX() - baseX);
    float deltaY = abs(movement.getY() - baseY);
    float deltaZ = abs(movement.getZ() - baseZ);
    float deltaRoll = abs(movement.getRoll() - baseRoll);
    float deltaPitch = abs(movement.getPitch() - basePitch);
    float deltaYaw = abs(movement.getYaw() - baseYaw);
    // ตรวจสอบว่ามีค่าใดเกินเกณฑ์หรือไม่
    bool motionDetected = (deltaX > MOTION_THRESHOLD ||
                           deltaY > MOTION_THRESHOLD ||
                           deltaZ > MOTION_THRESHOLD ||
                           deltaRoll > ROTATION_THRESHOLD ||
                           deltaPitch > ROTATION_THRESHOLD ||
                           deltaYaw > ROTATION_THRESHOLD);
}
```

```

// ----- State Machine การตรวจจับ -----
if (motionDetected && !inMotion) {
    // เริ่มเคลื่อนไหวใหม่
    inMotion = true;
    motionStartTime = millis();
    Monitor.println("👉 ตรวจพบการเคลื่อนไหว!");
    // ระบุประเภทการเคลื่อนไหว
    if (deltaRoll > ROTATION_THRESHOLD || deltaPitch > ROTATION_THRESHOLD) {
        Monitor.println("ประเภท: การหมุน");
    }
    if (deltaX > MOTION_THRESHOLD || deltaY > MOTION_THRESHOLD) {
        Monitor.println("ประเภท: การเคลื่อนที่เชิงเส้น");
    }
    if (deltaZ > MOTION_THRESHOLD * 2) {
        Monitor.println("ประเภท: การเคลื่อนที่ในแนวตั้ง");
    }
}
else if (!motionDetected && inMotion) {
    // อาจหยุดเคลื่อนไหวแล้ว
    if (stillStartTime == 0) {
        stillStartTime = millis(); // เริ่มจับเวลานิ่ง
    }
    else if (millis() - stillStartTime > STILL_TIMEOUT) {
        // นิ่งครบเวลาที่กำหนด → ถือว่าหยุดเคลื่อนไหว
        inMotion = false;
        unsigned long duration = (millis() - motionStartTime) / 1000;
        Monitor.print("👋 การเคลื่อนไหวหยุดลง ระยะเวลา: ");
        Monitor.print(duration);
        Monitor.println(" วินาที");
        stillStartTime = 0;
    }
}
else if (motionDetected && inMotion) {
    // ยังเคลื่อนไหวอยู่ รีเซ็ตตัวจับเวลานิ่ง
    stillStartTime = 0;
}
}

```



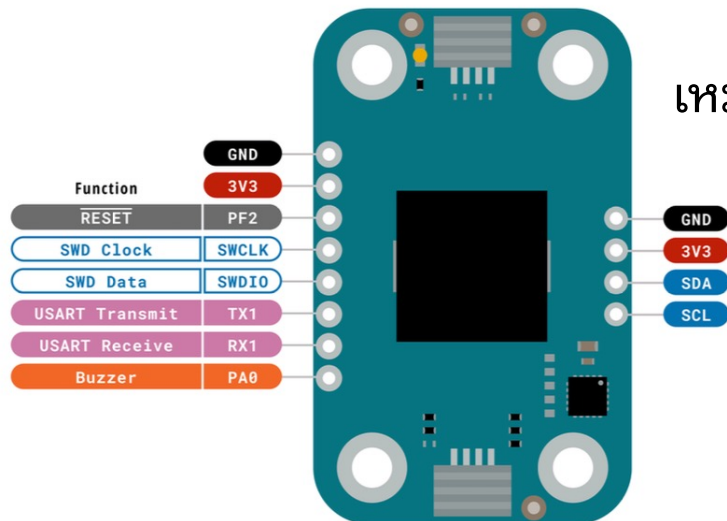
Modulino® Buzzer เป็นโมดูลสร้างสัญญาณเสียงอัจฉริยะในซีรีส์ Modulino® ที่มีขนาดกะทัดรัดและออกแบบประสงค์ ออกแบบมาเพื่อเพิ่มฟังก์ชันการแจ้งเตือนด้วยเสียงให้กับโปรเจกต์ต่าง ๆ

Modulino® Buzzer ใช้ไมโครคอนโทรลเลอร์ **STM32C011F4** บนบอร์ด ให้รูปแบบโมดูลขนาดกะทัดรัดและออกแบบประสงค์สำหรับสร้างเสียงเตือนและโทนเสียงต่าง ๆ ผ่านการควบคุมแบบ I²C เหมาะสำหรับการแจ้งเตือนด้วยเสียง ระบบสัญญาณเตือนภัย หรือการใช้งานด้านดนตรีอย่างง่ายในโปรเจกต์ IoT และงานเมกเกอร์

Modulino® **BUZZER**

ค่าเริ่มต้น (Default) คือ **0x3C**

เหมาะอย่างยิ่งสำหรับ:



- การแจ้งเตือนด้วยเสียง (Audio alerts): ในโปรเจกต์ที่ต้องการสถานะตอบรับเป็นเสียง
- ระบบสัญญาณเตือนภัย (Alarm systems): สำหรับอุปกรณ์ความปลอดภัยขนาดเล็ก
- งานดนตรีอย่างง่าย: การสร้างเมโลดี้พื้นฐานในโปรเจกต์ IoT หรือเครื่องเล่นเกมทำเอง

Key Functions (ฟังก์ชันหลัก)

- **tone(frequency, duration)** : สร้างเสียงตามความถี่ที่กำหนด (หน่วยเฮิรตซ์ Hz) และกำหนดระยะเวลาเล่นเสียง (หน่วยมิลลิวินาที)
- การตั้งค่า frequency เป็น 0 จะเป็นการหยุดเสียงทันที

▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

```
// เรียกใช้ไลบรารีสำหรับอุปกรณ์ Modulino
#include <Arduino_Modulino.h>
// สร้างอ็อบเจกต์สำหรับควบคุม Buzzer
ModulinoBuzzer buzzer;
// กำหนดค่าความถี่ของเสียง (หน่วย Hz)
// 440 Hz คือเสียงโน้ต A มาตรฐาน
int frequency = 440;
// กำหนดระยะเวลาเล่นเสียง (หน่วยมิลลิวินาที)
// 1000 ms = 1 วินาที
int duration = 1000;
void setup(){
    // เริ่มต้นระบบ Modulino
    Modulino.begin();
    // เริ่มต้นการทำงานของ Buzzer
    buzzer.begin();
}
void loop(){
    // สั่งให้ Buzzer ส่งเสียงตามความถี่และเวลาที่กำหนด
    buzzer.tone(frequency, duration);
    // หน่วงเวลา 1 วินาที
    delay(1000);
    // สั่งหยุดเสียง (ตั้งค่า frequency = 0)
    buzzer.tone(0, duration);
    // หน่วงเวลา 1 วินาทีก่อนเริ่มรอบใหม่
    delay(1000);
}
```

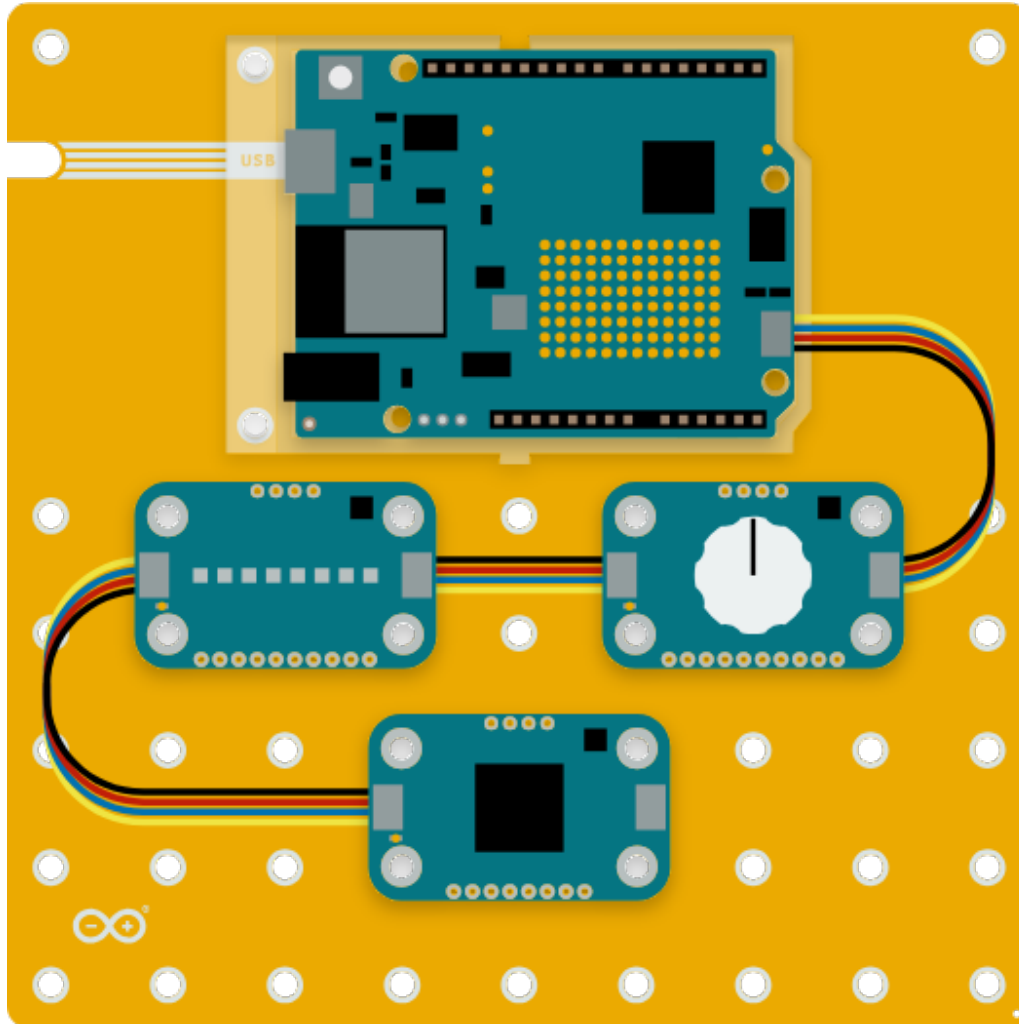
▼ Sketch Libraries

📖 Arduino_Modulino 0.7.0

```
// เรียกใช้ไลบรารี Modulino
#include <Arduino_Modulino.h>
// สร้างอ็อบเจกต์สำหรับควบคุม Buzzer
ModulinoBuzzer buzzer;
// ===== ตารางความถี่ =====
// กำหนดค่าความถี่ของโน้ตแต่ละตัว (หน่วย Hz)
// เช่น NOTE_A4 = 440 Hz
// REST คือการหยุดเสียง (ค่า 0)
#define NOTE_B0 31
#define NOTE_C1 33
#define NOTE_CS1 35
#define NOTE_D1 37
#define NOTE_DS1 39
#define NOTE_E1 41
#define NOTE_F1 44
#define NOTE_FS1 46
#define NOTE_G1 49
#define NOTE_GS1 52
#define NOTE_A1 55
#define NOTE_AS1 58
#define NOTE_B1 62
#define NOTE_C2 65
#define NOTE_CS2 69
#define NOTE_D2 73
#define NOTE_DS2 78
#define NOTE_E2 82
#define NOTE_F2 87
#define NOTE_FS2 93
#define NOTE_G2 98
#define NOTE_GS2 104
#define NOTE_A2 110
#define NOTE_AS2 117
#define NOTE_B2 123
#define NOTE_C3 131
#define NOTE_CS3 139
#define NOTE_D3 147
#define NOTE_DS3 156
#define NOTE_E3 165
#define NOTE_F3 175
#define NOTE_FS3 185
#define NOTE_G3 196
#define NOTE_GS3 208
#define NOTE_A3 220
#define NOTE_AS3 233
#define NOTE_B3 247
#define NOTE_C4 262
#define NOTE_CS4 277
#define NOTE_D4 294
#define NOTE_DS4 311
#define NOTE_E4 330
```

```
#define NOTE_F4 349
#define NOTE_FS4 370
#define NOTE_G4 392
#define NOTE_GS4 415
#define NOTE_A4 440
#define NOTE_AS4 466
#define NOTE_B4 494
#define NOTE_C5 523
#define NOTE_CS5 554
#define NOTE_D5 587
#define NOTE_DS5 622
#define NOTE_E5 659
#define NOTE_F5 698
#define NOTE_FS5 740
#define NOTE_G5 784
#define NOTE_GS5 831
#define NOTE_A5 880
#define NOTE_AS5 932
#define NOTE_B5 988
#define NOTE_C6 1047
#define NOTE_CS6 1109
#define NOTE_D6 1175
#define NOTE_DS6 1245
#define NOTE_E6 1319
#define NOTE_F6 1397
#define NOTE_FS6 1480
#define NOTE_G6 1568
#define NOTE_GS6 1661
#define NOTE_A6 1760
#define NOTE_AS6 1865
#define NOTE_B6 1976
#define NOTE_C7 2093
#define NOTE_CS7 2217
#define NOTE_D7 2349
#define NOTE_DS7 2489
#define NOTE_E7 2637
#define NOTE_F7 2794
#define NOTE_FS7 2960
#define NOTE_G7 3136
#define NOTE_GS7 3322
#define NOTE_A7 3520
#define NOTE_AS7 3729
#define NOTE_B7 3951
#define NOTE_C8 4186
#define NOTE_REST 0
```

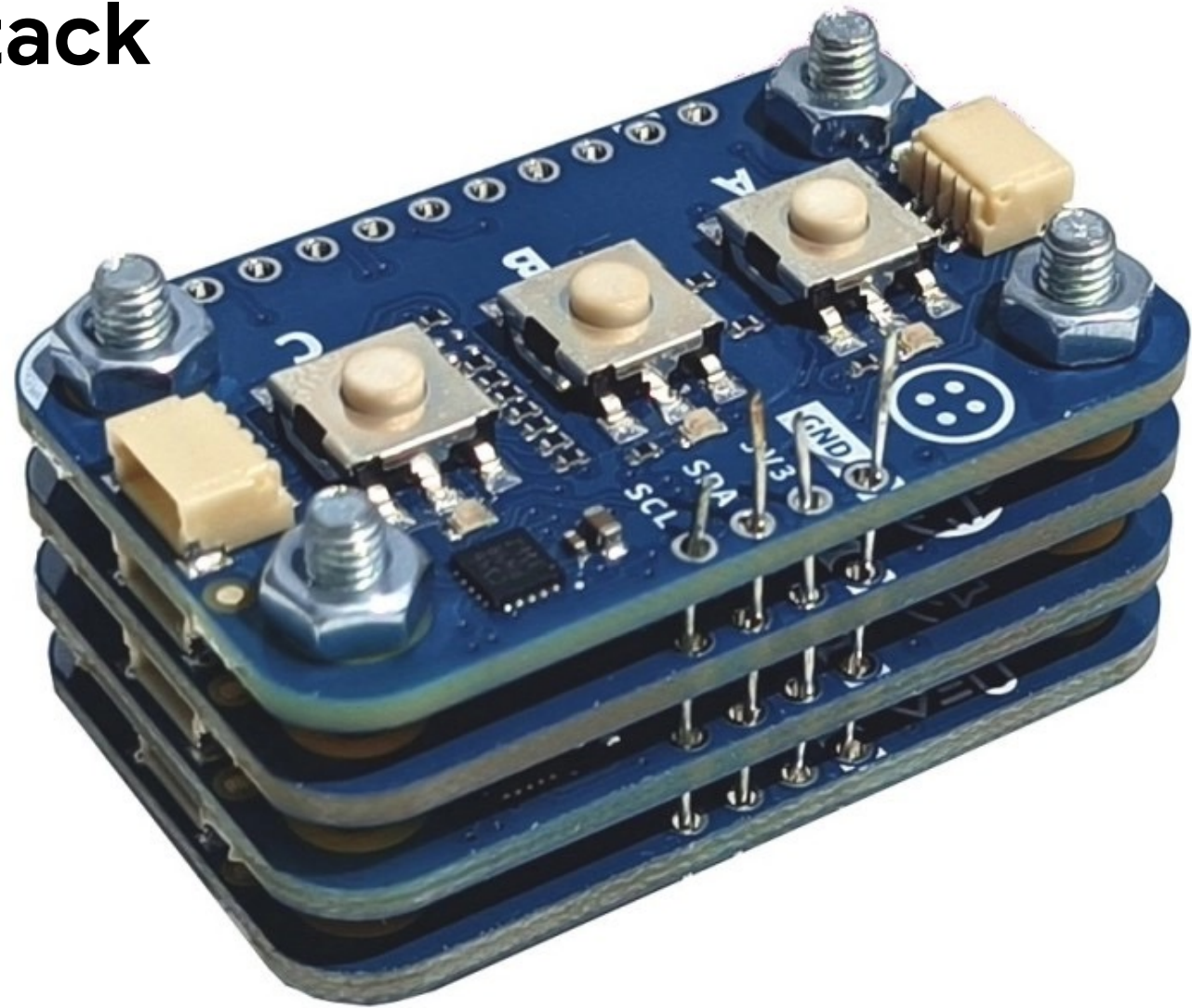
```
// ===== กำหนดจังหวะเพลง =====
int tempo = 200; // BPM (ยิ่งมาก เพลงยิ่งเร็ว)
// ===== ข้อมูลเมโลดี้ =====
// เก็บข้อมูลเป็นคู่ ๆ [โน้ต, ค่าจังหวะ]
// ค่าจังหวะ เช่น 4 = ตัวดำ, 8 = ตัวเซมิโน้ต
// ถ้าเป็นค่าติดลบ หมายถึง dotted note (เพิ่มเวลา 50%)
int melody[] = {
    NOTE_E5,8, NOTE_E5,8, REST,8, NOTE_E5,8, REST,8, NOTE_C5,8, NOTE_E5,8,
    NOTE_G5,4, REST,4, NOTE_G4,8, REST,4,
    // (ท่อนเพลงต่อเนื่อง...)
    NOTE_G4,8, NOTE_D4,8, NOTE_E4,-2
};
// คำนวณจำนวนโน้ตทั้งหมด (หาร 2 เพราะเก็บเป็นคู่)
int notes = sizeof(melody) / sizeof(melody[0]) / 2;
void setup() {
    Modulino.begin(); // เริ่มต้นระบบ Modulino
    buzzer.begin(); // เริ่มต้นการทำงานของ Buzzer
}
void loop() {
    // คำนวณระยะเวลา 1 ห้องเพลง (ตัวกลม)
    int wholenote = (60000 * 4) / tempo;
    // วนลูปเล่นโน้ตทั้งหมด
    for (int i = 0; i < notes * 2; i += 2) {
        int noteDuration = melody[i + 1]; // ค่าจังหวะ
        int duration; // เวลาที่จะเล่นจริง
        // ถ้าเป็นโน้ตปกติ
        if (noteDuration > 0) {
            duration = wholenote / noteDuration;
        }
        // ถ้าเป็น dotted note (ค่าติดลบ)
        else {
            duration = (wholenote / abs(noteDuration)) * 1.5;
        }
        // ถ้าเป็น REST ให้หน่วงเวลาเฉย ๆ
        if (melody[i] == REST) {
            delay(duration);
        }
        else {
            // เล่นเสียงตามความถี่ที่กำหนด
            buzzer.tone(melody[i], duration);
            // หน่วงเวลาเล็กน้อยให้โน้ตแยกจากกัน
            delay(duration * 1.3);
            // หยุดเสียงสั้น ๆ
            buzzer.tone(0, 10);
        }
    }
    // เมื่อเล่นจบทั้งเพลง ให้พัก 5 วินาทีก่อนเริ่มใหม่
    delay(5000);
}
```

Modulino ทำงานอย่างไร?

- โหนด Modulino เชื่อมต่อกันเป็นสายโซ่ (daisy chain) ผ่านคอนเนกเตอร์ Qwiic
- ข้อมูลถูกส่งผ่านไปตามสายโซ่เดียวกันนี้
- ลำดับหรือทิศทางของโหนดในสายโซ่ไม่มีผลต่อการทำงาน
- เซนเซอร์และแอคชูเอเตอร์ที่มีคุณสมบัติต่างกันสามารถทำงานร่วมกันได้ โดยไม่ต้องเดินสายเพิ่มหรือเขียนโค้ดที่ซับซ้อน
- Modulino แต่ละตัวมีที่อยู่ (address) เฉพาะ ทำให้ระบบสามารถแยกแยะได้
- Modulino หลายรุ่นสามารถตั้งค่า address เองได้ จึงสามารถใช้โมดูลชนิดเดียวกันหลายตัวในสายโซ่เดียวกันได้

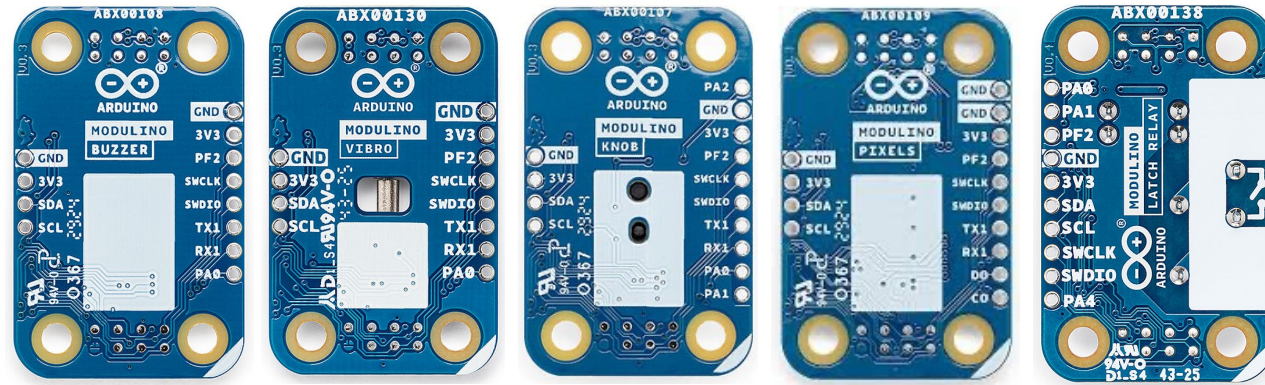
Modulino Stack



Technical Reference: I2C Addresses

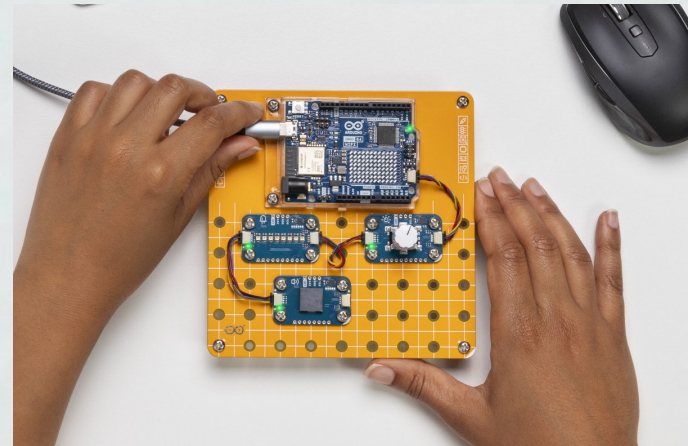
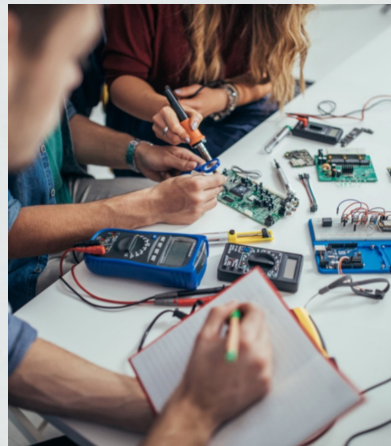
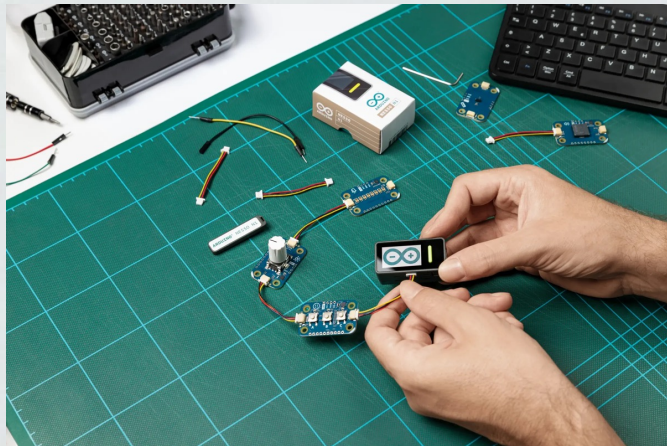
Module Name	Default Hex Address
Modulino® BUTTONS	0x7C
Modulino® KNOB	0x76
Modulino® PIXELS	0x6C
Modulino® BUZZER	0x3C
Modulino® THERMO	0x44
Modulino® DISTANCE	0x29
Modulino® MOVEMENT	0x6A

Modulino®



มีพื้นที่ White rectangle (สี่เหลี่ยมสีขาว)
ด้านหลังบอร์ดสำหรับจดบันทึกค่า Address ใหม่

Start Building Today



Modulino® เปลี่ยนความซับซ้อนของฮาร์ดแวร์ให้เป็นเรื่องง่าย
เพื่อให้คุณโฟกัสกับการเขียนโค้ดและสร้างสรรค์นวัตกรรม

Ready to code? Visit the [Arduino Library documentation](#).